

Desenvolvimento de Software Integrado - DevOps

Slides da disciplina | Turma 4 - Z251 | Semestre 2026.2

Plan

Code

Build

Test

Release

Deploy

Operate

Monitor

Sobre o Professor

Quem conduz esta disciplina



Arimatéia Júnior

Professor - Desenvolvimento de Software Integrado - DevOps

Formação



- Pós-graduado em Engenharia de Software com ênfase em DevOps — UNIFOR (2025).
- MBA em Arquitetura de Software e Soluções — XP Educação (2023).
- Especialização em Segurança da Informação — UNINASSAU (2021).
- Formado em Redes de Computadores — Faculdade Lourenço Filho (2014).

Experiência



- Arquiteto de Software e Soluções — TIVIT (cliente Petrobras).
- Consultor de Tecnologia — ARI JR Technology.

Certificações



- Microsoft, AWS, Google Cloud, Oracle e VMware.

Sobre mim

- Apaixonado por família, tecnologia e futebol.

Visão Geral da Disciplina

24h em seis encontros: cultura, automação, entrega, operação e IA aplicada

Objetivo da turma

- Conectar desenvolvimento, qualidade e operação em um fluxo único
- Trabalhar com artefatos reais: repositório, pipeline, containers, evidências e IA
- Fechar com projeto integrador e uso responsável de IA registrado

Eixo prático

- Diagnóstico + IA no SDLC
- Git, PR e revisão assistida
- CI, testes e qualidade
- Docker/Compose e troubleshooting
- CD, segurança, observabilidade e evidências

Resultado esperado

- Aplicação executável em ambiente reproduzível
- Pipeline com validações e evidências
- Uso de IA documentado e validado
- Release/rollback ou plano de rollback
- Evidências de operação e melhoria

Pergunta orientadora

- Como transformar uma aplicação em um produto entregável, observável, seguro e evolutivo usando DevOps e IA com validação humana?

Objetivos de Aprendizagem

O que você deve conseguir fazer ao final da disciplina

Compreender

- Explicar cultura DevOps e responsabilidade compartilhada.
- Identificar gargalos entre desenvolvimento, qualidade e operação.
- Relacionar métricas, feedback e melhoria contínua.

Construir

- Aplicar Git e PRs com revisão objetiva.
- Criar pipeline de CI com testes e artefatos.
- Containerizar uma aplicação e padronizar execução.
- Planejar CD com segurança básica e rollback.

Avaliar

- Usar IA criticamente no SDLC.
- Analisar logs, falhas, riscos e decisões.
- Defender o projeto final com evidências executáveis.

Linha do Tempo dos Encontros

Datas oficiais da turma

10/09



- Cultura DevOps, IA no SDLC e diagnóstico

11/09



- Git, colaboração, qualidade e IA assistida

12/09



- CI, testes automatizados e IA para automação

24/09



- Containers, integração e troubleshooting

25/09



- CD, configuração, segurança e governança de IA

26/09



- Observabilidade, IA aplicada e projeto

Projeto Integrador

Um fluxo DevOps completo em pequena escala



Artefatos mínimos

- README de execução
- Pipeline versionado
- Dockerfile e compose.yaml
- Evidências de build/teste/release
- Registro do uso de IA quando aplicado

Critérios técnicos

- Automação rastreável
- Falha visível e tratável
- Ambiente reproduzível
- Rollback ou plano de rollback
- Sugestões de IA validadas por teste e revisão

Defesa final

- Diagnóstico inicial
- Decisões técnicas
- Uso crítico de IA
- Tradeoffs e limitações
- Evolução proposta

Projeto Integrador - Incrementos por Encontro

Um único projeto evolui ao longo dos seis encontros

Incrementos por encontro

- E1: diagnóstico e proposta priorizada.
- E2: organização, equipe e código-base na main.
- E3: pipeline CI com testes.
- E4: execução com Docker/Compose.
- E5: release, rollback e segurança.
- E6: observabilidade e defesa.

Critério de escolha da aplicação

- Aplicação pequena, mas realista.
- Deve ter build, teste e execução local.
- Ideal ter API + dependência externa ou banco.
- Evitar stack complexa que consuma o tempo disponível.

Evidência mínima

- README de execução.
- Print/log de pipeline.
- Dockerfile e compose.yaml.
- Registro de release/rollback.
- Registro de uso de IA, quando aplicado.

Ferramentas e Ambiente

O que ter pronto antes do primeiro encontro

Essencial



- Git instalado e configurado.
- Conta em GitHub, GitLab ou Azure DevOps.
- Editor com terminal integrado.
- Docker Desktop ou Docker Engine + Compose.

Pipeline



- GitHub Actions é a opção mais simples para laboratório.
- GitLab CI ou Azure Pipelines seguem a mesma lógica.
- Exemplos curtos e versionados no repositório da turma.

IA assistida



- GitHub Copilot, ChatGPT/Codex, Cursor, Gemini Code Assist, Amazon Q ou equivalente.
- Acesso opcional, com demonstrações em sala.

Avaliação

Distribuição final alinhada ao plano de ensino

25% Projeto funcional

Repositório organizado, aplicação executável, documentação e registro crítico de IA

25% Pipeline CI

Build, testes, qualidade, análise de falhas e artefatos/imagens publicados

20% Containers/Compose

Dockerfile, compose.yaml, variáveis, volumes e health checks

15% Entrega e operação

Release, rollback/plano, segurança básica, logs, métricas e governança de IA

10% Diagnóstico DevOps

Gargalos, riscos, oportunidades de IA e proposta de evolução da automação

5% Apresentação final

Defesa técnica das decisões, uso de IA, tradeoffs e próximos passos

Leitura da rubrica: a IA pode apoiar o trabalho, mas a nota depende de evidências executáveis, validação humana, testes e decisões técnicas defendidas.

IA Aplicada ao SDLC

Onde usar durante o fluxo DevOps sem perder rigor técnico

Planejar e entender



- Refinar histórias e critérios de aceite
- Resumir contexto técnico do repositório
- Mapear riscos, dependências e premissas

Codificar e refatorar



- Sugerir implementação incremental
- Explicar código legado ou desconhecido
- Propor refatoração pequena e revisável

Testar e revisar



- Gerar casos de teste e dados de borda
- Revisar diff/PR com checklist explícito
- Conferir cobertura, legibilidade e regressão

Automatizar pipeline



- Explicar YAML de CI/CD
- Sugerir correção para falha de job
- Documentar etapas e variáveis necessárias

Operar e observar



- Sumarizar logs e levantar hipóteses
- Relacionar métrica, erro e deploy recente
- Gerar runbook inicial para incidentes

Documentar decisão



- Produzir README e release notes
- Registrar prompts, saídas e validações
- Explicar o que foi aceito, rejeitado e por quê

Uso Seguro de IA

Regras práticas para laboratório, projeto e avaliação

Antes de perguntar

- Não enviar segredos, tokens, chaves, dados pessoais ou código sensível
- Fornecer contexto mínimo: objetivo, stack, erro, restrições e trecho relevante
- Pedir plano curto ou hipóteses antes de aceitar código pronto

Durante a resposta

- Checar se a solução respeita arquitetura, dependências e padrões do projeto
- Tratar saída da IA como sugestão, não como fonte de verdade
- Investigar alucinações, comandos perigosos e permissões excessivas

Depois de aplicar

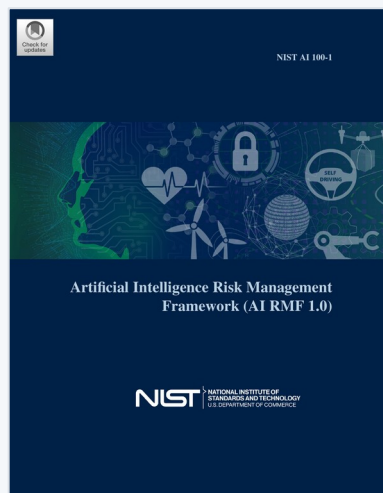
- Rodar testes, lint, build e validação manual relevante
- Revisar segurança: segredos, entrada de usuário, dependências e logs
- Manter diff pequeno para facilitar revisão humana

Evidência para avaliação

- Registrar prompt/contexto, saída útil, alterações aceitas e descartadas
- Explicar validações feitas e correções humanas realizadas
- Citar limites, riscos e próximos passos quando a IA foi usada

Aprofundamento em Segurança de IA

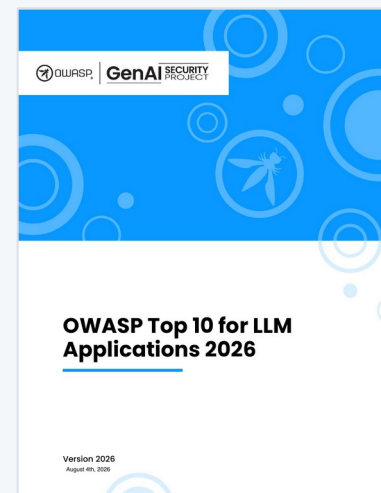
Leitura complementar disponível nos materiais do EAD desta disciplina



NIST AI RMF 1.0

Artificial Intelligence Risk Management Framework

- Framework de referência para gerenciar riscos de sistemas de IA.
- Base para pensar governança, confiabilidade e uso responsável no projeto integrador.



OWASP Top 10 for LLM Applications 2026

OWASP GenAI Security Project

- Os 10 riscos mais críticos em aplicações que usam LLM.
- Referência direta para prompt injection, vazamento de dados e alucinação.

Disponível na área de materiais de apoio do EAD — leitura recomendada antes do Encontro 5.

DORA 2026: O Que os Dados Dizem Sobre IA em DevOps

DORA (DevOps Research and Assessment)

- Programa de pesquisa do Google Cloud que estuda o desempenho de equipes de engenharia de software e identifica as práticas capazes de impulsionar a entrega e as operações de tecnologia.

Antes de usar IA em DevOps, veja o que a pesquisa mais citada da área já mediu

Adoção

- 90% dos profissionais de tecnologia já usam IA no trabalho.
- Mais de 80% relatam ganho de produtividade individual.

Efeito colateral

- A cada 25% de aumento na adoção de IA, throughput de entrega cai 1,5%.
- Estabilidade de entrega cai 7,2% no mesmo cenário.

Paradoxo da produtividade

- Incidentes por Pull Request subiram 242,7%.
- O tempo economizado na criação de código é realocado para auditoria e verificação.

Conclusão do DORA

- IA é um amplificador, não substituto de boa engenharia.
- Só expande capacidade em times com testes sólidos, PRs pequenos e fluxo saudável.



Encontro 1

10/09/2026 | Cultura DevOps, IA no SDLC e diagnóstico

Encontro 1 - Cultura e Diagnóstico

Base: cultura DevOps + diagnóstico de maturidade

1. Abertura e contexto

- Objetivo da disciplina, conhecimentos prévios e projeto integrador.
- Papel da IA como apoio, não substituição da engenharia.

2. Cultura e ciclo

- Dev x Ops e responsabilidade compartilhada
- Plan, Code, Build, Test, Release, Deploy, Operate, Monitor
- Onde IA apoia análise, código, testes, operação e feedback

3. Diagnóstico

- Quick check em grupos
- Mapear gargalos e tarefas manuais
- Identificar oportunidades e riscos de IA
- Registrar hipóteses de melhoria

Saída do encontro

- Documento curto: diagnóstico DevOps + uma proposta priorizada, incluindo oportunidade de IA e validação necessária quando fizer sentido.

Conceitos Essenciais - DevOps

O que sustenta a prática do encontro

C

Cultura

Foco nas pessoas, na colaboração e na responsabilidade compartilhada entre os times.

A

Automação

Eliminar tarefas manuais repetitivas para evitar erros e acelerar processos.

L

Lean

Usar princípios enxutos para eliminar desperdícios e focar no que gera valor real para o cliente.

M

Medição

Coletar dados e métricas (como as métricas DORA) para entender se os processos estão melhorando.

S

Compartilhamento

Dividir conhecimento, sucessos e erros entre toda a organização.

DevOps em Uma Frase: Mensagem, Modelos e Métricas

Como amarrar CALMS ao vocabulário de decisão do dia a dia

Mensagem central

- DevOps não é uma ferramenta.
- É integração entre cultura, processo, automação e medição.
- O objetivo é reduzir tempo de ciclo sem perder qualidade e segurança.

Modelos úteis

- CALMS: cultura, automação, lean, medição e compartilhamento.
- Fluxo de valor: do pedido ao software em produção.
- Feedback rápido: erro descoberto cedo é mais barato.

Métricas para citar

- Lead time for changes.
- Deployment frequency.
- Change failure rate.
- Mean time to restore.
- Use como linguagem de decisão, não como decoração.

As 4 Métricas DORA

Como medir a saúde da entrega de software

Deployment Frequency

- Com que frequência o time entrega em produção.
- Referência de elite: sob demanda, múltiplas vezes ao dia.

Lead Time for Changes

- Tempo do commit até rodar em produção.
- Referência de elite: menos de uma hora.

Change Failure Rate

- Percentual de mudanças que causam falha em produção.
- Referência de elite: entre 0% e 15%.

Time to Restore (MTTR)

- Tempo para recuperar de uma falha em produção.
- Referência de elite: menos de uma hora.

DORA na Prática: Benchmarks Reais de Mercado

Números públicos e verificáveis, não estimativa

Amazon

- Deploy a cada 11,7 segundos em média (~50 milhões de deploys/ano).
- Dado popularizado por Gene Kim, autor de "The DevOps Handbook" — bibliografia oficial da disciplina.

Etsy

- Saiu de 2 deploys por semana para mais de 50 deploys por dia após automatizar o pipeline.
- Um dos casos mais citados de transformação DevOps na literatura.

iFood e Netflix

- iFood: 1.500+ engenheiros, ~10 mil repositórios, deploy canário automatizado (Canary 2.0).
- Netflix: milhares de deploys por dia.

Por Que o DevOps Surgiu

Origem e contexto do movimento

O muro da confusão

- Dev quer lançar rápido; Ops quer estabilidade — metas em conflito.
- Cada lado media sucesso de um jeito diferente e se culpava pelos problemas.
- Passagem de bastão sem responsabilidade compartilhada gerava atrito.

Raízes do movimento

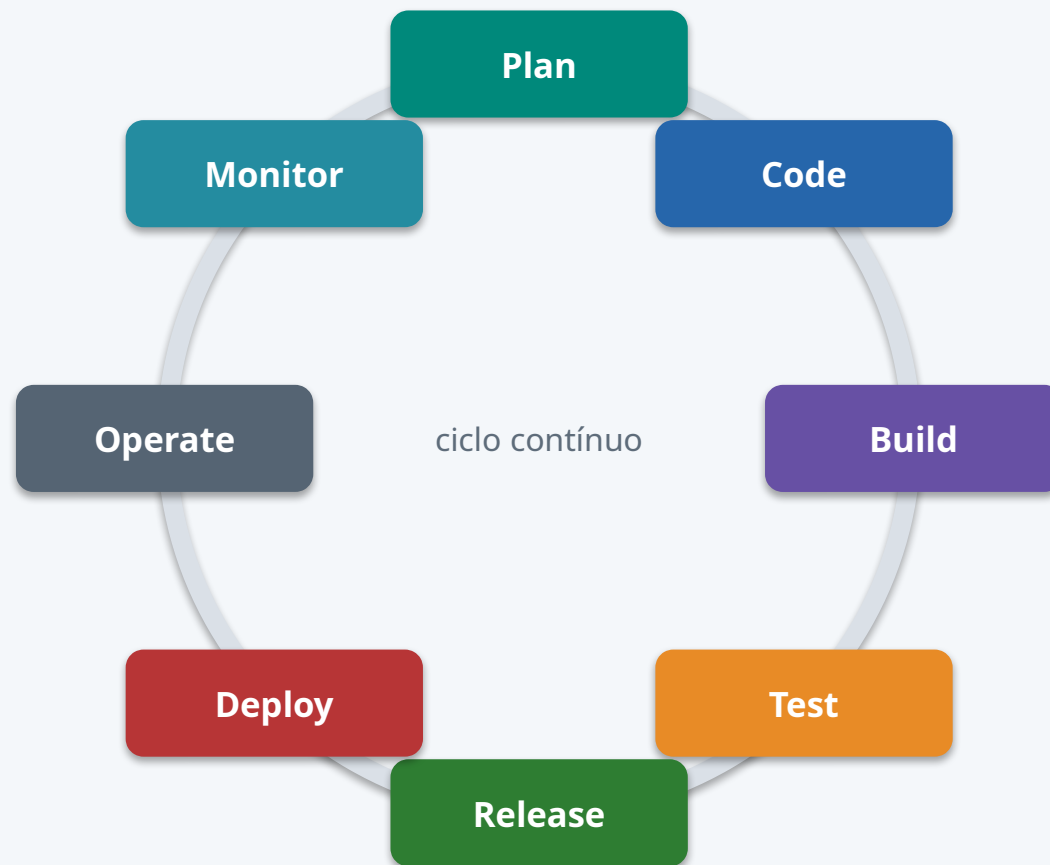
- Agile (2001): entrega iterativa e feedback frequente.
- Lean: eliminar desperdício, otimizar o fluxo de valor.
- Gestão de operações: disciplina de mudança e confiabilidade.

Mudança de mentalidade

- De “jogar por cima do muro” para “you build it, you run it”.
- Responsabilidade compartilhada do código até a operação em produção.

O Ciclo DevOps

Um loop contínuo de feedback, não uma linha reta



Monitor alimenta o próximo Plan: feedback rápido é o que fecha o loop e sustenta a melhoria contínua.

O Ciclo DevOps: Cada Etapa em Detalhe

Um loop contínuo, não uma linha reta

Loop de desenvolvimento

- Plan: entender o problema e definir critérios de aceite.
- Code: implementar em mudanças pequenas e revisáveis.
- Build: compilar, empacotar e testar automaticamente.

Loop de operação

- Release: preparar e aprovar uma versão para entrega.
- Deploy: colocar a versão no ambiente de destino.
- Operate/Monitor: manter saudável e observar o comportamento real.

O ponto de conexão

- Monitor alimenta o próximo Plan — é um ciclo, não uma linha.
- Feedback rápido é o que fecha o loop e sustenta a melhoria contínua.

Sinais de que um Time Precisa de DevOps

Para provocar debate: quais desses sintomas vocês já viveram?

Sintomas comuns

- Deploys manuais, longos e concentrados em uma pessoa.
- Medo de lançar em sexta-feira.
- “Funciona na minha máquina”, mas não em produção.
- Falta de rastreabilidade: ninguém sabe o que mudou entre versões.

Custos escondidos

- Tempo perdido coordenando quem pode mudar o quê e quando.
- Bugs descobertos tarde custam muito mais para corrigir.
- Burnout de quem sempre “segura” a produção sozinho.

Pergunta para discussão

- Quais desses sintomas vocês já viveram — como dev, como usuário ou como cliente?
- O que mudaria se a responsabilidade fosse compartilhada desde o início?

IA no SDLC: Onde Ajuda e Onde Exige Cautela

Ganhos reais e riscos reais, lado a lado

Ganhos reais

- Reduz tempo em tarefas repetitivas: boilerplate, testes, documentação.
- Acelera entendimento de código legado e análise de logs.
- Ajuda a levantar hipóteses e alternativas técnicas rapidamente.

Riscos reais

- Alucinação de comandos, APIs ou flags que não existem.
- Dependência intelectual: aceitar sem entender o que foi gerado.
- Vazamento de dado sensível quando colado sem cuidado no prompt.

Regra prática da disciplina

- Toda saída de IA é hipótese até ser validada por teste, execução ou revisão humana.
- Quem usa precisa conseguir explicar e defender o resultado.

Atividade 1 - Diagnóstico DevOps

Como fazer, em 3 passos

Passo 1 — Mapear o fluxo

- Escolha uma demanda simples e real (do dia a dia do grupo ou de um sistema que vocês conhecem).
- Liste as etapas até a entrega, marcando esperas, retrabalho e aprovações manuais.

Passo 2 — Classificar gargalos

- Para cada gargalo encontrado, classifique em cultura, processo ou ferramenta/dado.
- Seja específico: "sistema lento" não é gargalo; "deploy manual sem revisão" é.

Passo 3 — Priorizar

- Escolha 3 gargalos e registre impacto, esforço, risco e evidência para cada um.
- Escolha 1 melhoria final, indicando onde a IA ajuda e onde pode aumentar o risco.

Antes de começar

- Grupos de 4 a 5 integrantes, já formados no início do encontro.

Atividade 1 - Diagnóstico DevOps

Perguntas guia, evidências e entrega

Perguntas guia

- Onde há espera manual?
- Onde falta feedback?
- Que falha só aparece tarde?
- Quem é dono da operação?
- Onde IA reduz esforço sem ocultar risco?

Evidências

- Mapa simples do fluxo atual
- Lista de gargalos
- Risco por gargalo
- Oportunidade de IA e validação necessária
- Métrica que mostraria melhoria

Entrega

- 1 página por grupo
- Priorizar 3 melhorias
- Classificar em cultura, processo, ferramenta, dado ou IA

Conexão com avaliação

- Esse diagnóstico vira insumo para a proposta de implantação/evolução DevOps do projeto final.

Prompts Úteis - Diagnóstico

Sem dados sensíveis no conteúdo do prompt



Mapear fluxo

- Atue como consultor DevOps. Com base no fluxo abaixo, identifique gargalos, riscos e pontos de automação.
- Fluxo: <descrever sem dados sensíveis>.
- Responda em tabela: etapa, gargalo, impacto, evidência, melhoria.

Criticar melhoria

- Avalie esta proposta de melhoria DevOps. Aponte premissas frágeis, riscos operacionais e como medir se funcionou.
- Proposta: <texto>.

Definir métrica

- Para o gargalo abaixo, sugira uma métrica simples, como coletar a evidência e qual decisão essa métrica apoiaria.
- Gargalo: <texto>.

Depois do Encontro 1

O que levar pronto para o Encontro 2: Git e colaboração

Tarefa

- Refinar a proposta de melhoria escolhida.
- Escolher a aplicação e a stack do grupo.
- Levar ao próximo encontro: nome, stack e escopo mínimo para criar o código-base.

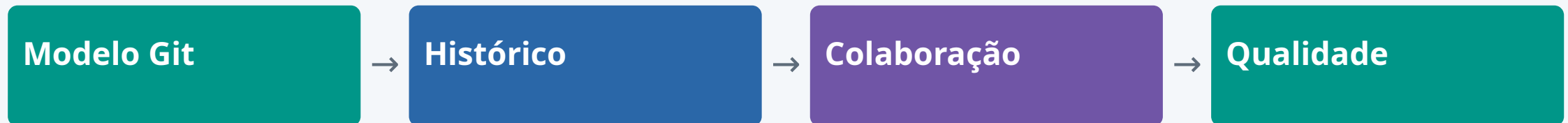


Encontro 2

11/09/2026 | Git, colaboração, qualidade e IA assistida

Encontro 2 — Git, Colaboração, Qualidade e IA Assistida

11/09/2026 • Do trabalho individual à mudança revisada



Objetivo: transformar código individual em um fluxo colaborativo, rastreável e revisável, com Pull Requests e assistência crítica de IA.

Entender → comparar exemplos → discutir → iniciar o projeto

Git x GitHub: Não São a Mesma Coisa

Ferramenta de versionamento e plataforma de colaboração

Git • no computador

Histórico distribuído

Commits, branches e merges

Funciona sem GitHub

GitHub • remoto

Hospeda repositórios Git

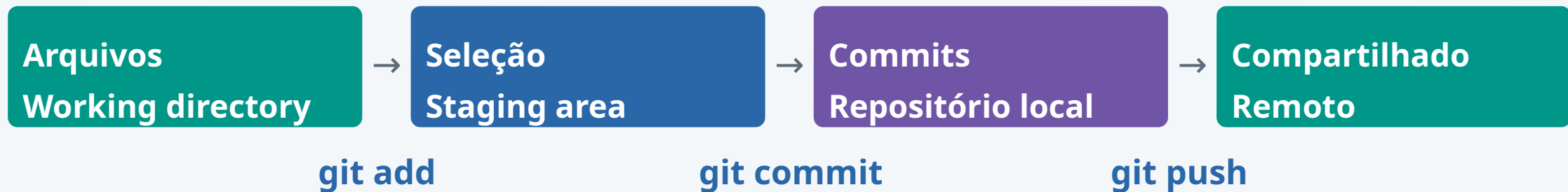
PRs, Issues, Actions e Releases

Acesso e colaboração

Git controla versões. GitHub organiza a colaboração em torno do repositório.

O Fluxo Local do Git

Três áreas locais; a publicação é uma etapa separada



Edite arquivos → selecione o que entra na próxima versão → registre o commit → publique os commits.

git add e git commit não enviam alterações ao GitHub.

Do Arquivo Alterado ao Commit

Inspecione antes de registrar

Antes do staging

git status: estado dos arquivos

git diff: alterações fora do staging

Antes do commit

git diff --staged: revisão do staging

git add: seleciona conteúdo, não move o arquivo

```
git status
git diff
git add README.md
git commit -m "docs: adiciona instruções"
```

Um commit deve expressar uma mudança coerente.

Git Local x Git Remoto

Branches locais e referências de acompanhamento



```
git remote -v  
git branch  
git branch -r  
git branch -a
```

origin é um apelido configurável para um remoto.

O Que é origin/main?

O último estado conhecido do remoto está guardado localmente

No computador

main → seu trabalho local

origin/main → estado conhecido

Ambas podem apontar para commits diferentes.

No GitHub

main → branch do servidor

Pode avançar enquanto você está offline.

fetch atualiza a referência local.

origin/main é uma referência local; não é uma consulta ao GitHub em tempo real.

Fetch, Pull e Push

Três ações, três efeitos diferentes

fetch

Busca commits e atualiza referências remotas locais.
“Veja o que mudou.”

pull

Busca e integra na branch atual.
“Traga e integre comigo.”

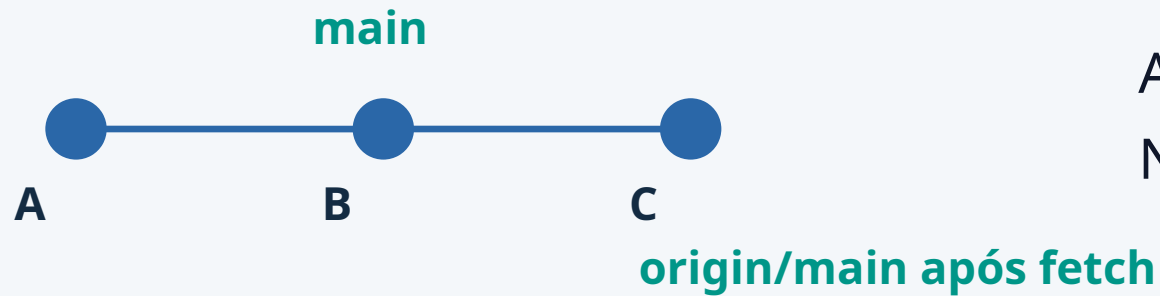
push

Envia commits e tenta atualizar referências no remoto.
“Publique o que fiz.”

Antes de integrar, saiba qual branch está selecionada.

O Que o git fetch Faz?

Remoto avançou até C; sua main continua em B

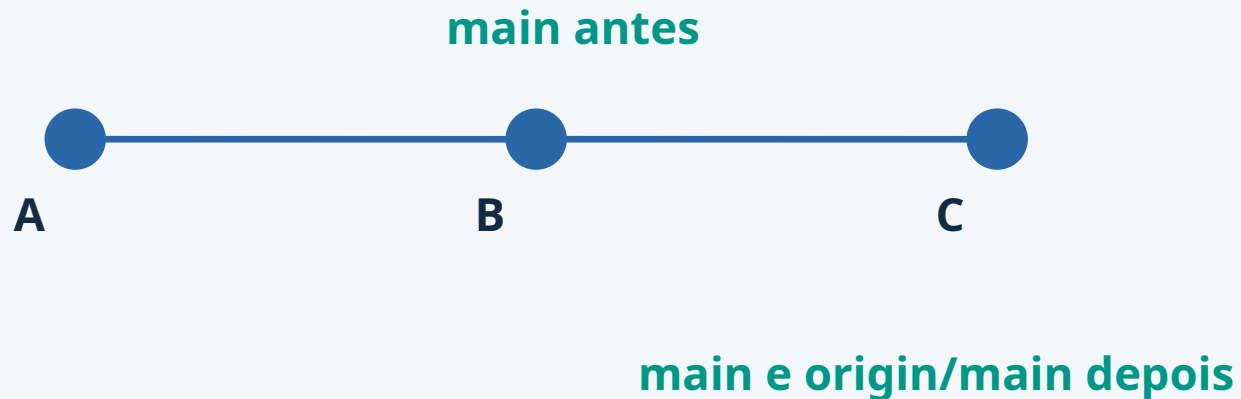


```
git fetch origin
git log main..origin/main
git diff main origin/main
```

fetch atualiza origin/main sem mover automaticamente a main.

O Que o git pull Faz?

Busca + integração; neste exemplo, avanço fast-forward



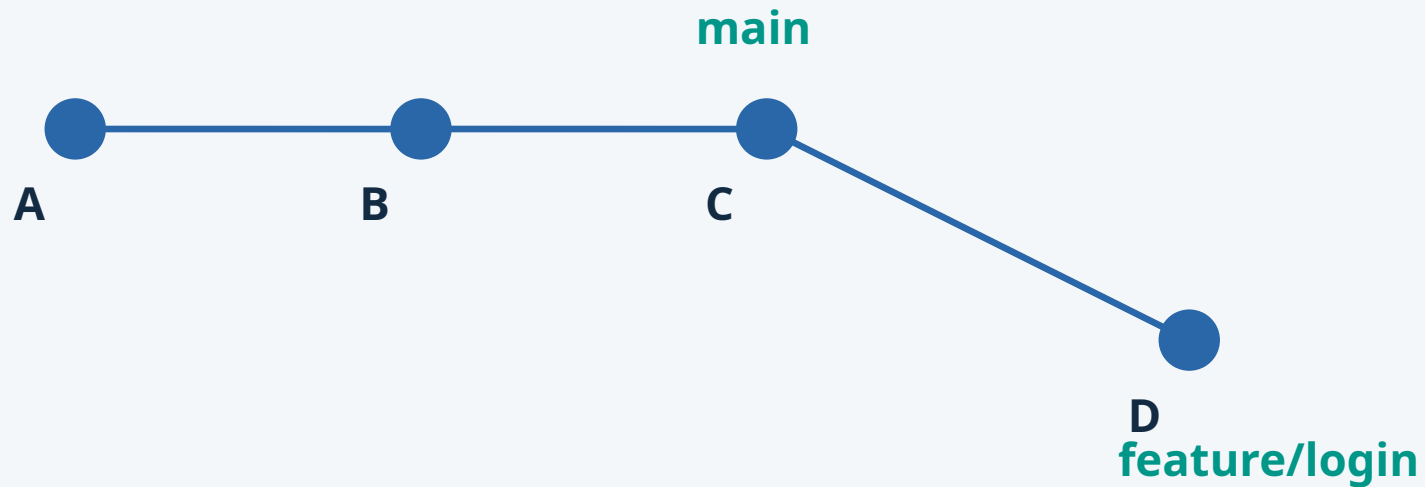
pull pode integrar por merge ou rebase, conforme opções e configuração.

```
git switch main
git pull --ff-only origin main
```

--ff-only recusa a atualização se os históricos divergiram.

O Que é uma Branch?

Uma referência móvel para um commit



Ao criar: ambas em C.
Após commitar: só a feature avança.

```
git switch -c feature/login
```

Criar uma branch não duplica todos os arquivos do projeto.

O Que é HEAD?

Em uma branch, HEAD é o ponteiro que mostra onde estamos no Git neste momento



```
git branch
* feature/login
main
```

Exceção: em detached HEAD, HEAD aponta diretamente para um commit.

O asterisco em git branch identifica a branch selecionada.

Como Enxergar o Histórico do Git

Leia o hash, a intenção e as referências

```
git log
git log --oneline
git log --oneline --graph --decorate --all
```

```
* 92ab341 (HEAD -> feature/login) feat: adiciona login
* 73eab21 docs: atualiza README
* c82d110 (origin/main, main) chore: projeto inicial
```

O grafo mostra relações entre commits; os rótulos indicam referências.

Merge: Duas Situações Diferentes

O formato do histórico define como as linhas podem ser integradas

Fast-forward

main não avançou por outro caminho.
Basta mover sua referência até a feature.
Nenhum commit de junção é necessário.

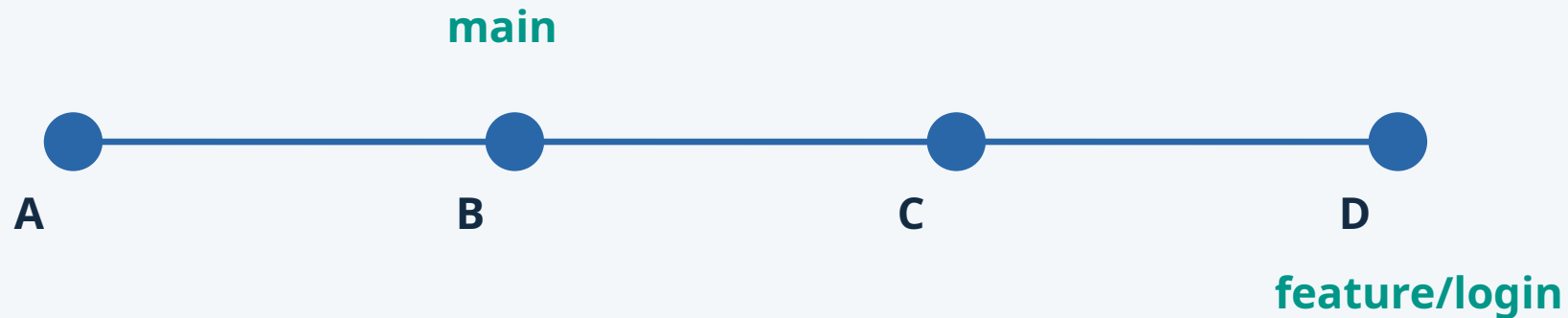
Merge de três vias

As branches divergiram.
Git considera a base comum e as duas pontas.
A integração cria um commit com dois pais.

Pergunta-chave: a main está no mesmo caminho de commits da feature, sem ter avançado separadamente ?

Fast-forward — Antes da Integração

A feature avançou; a main continua em B



C e D implementam login. Nenhum commit exclusivo foi adicionado à main.

Todos os commits da main já fazem parte do histórico da feature.

Fast-forward — Depois da Integração

O conteúdo passa a incluir a feature; os commits existentes permanecem

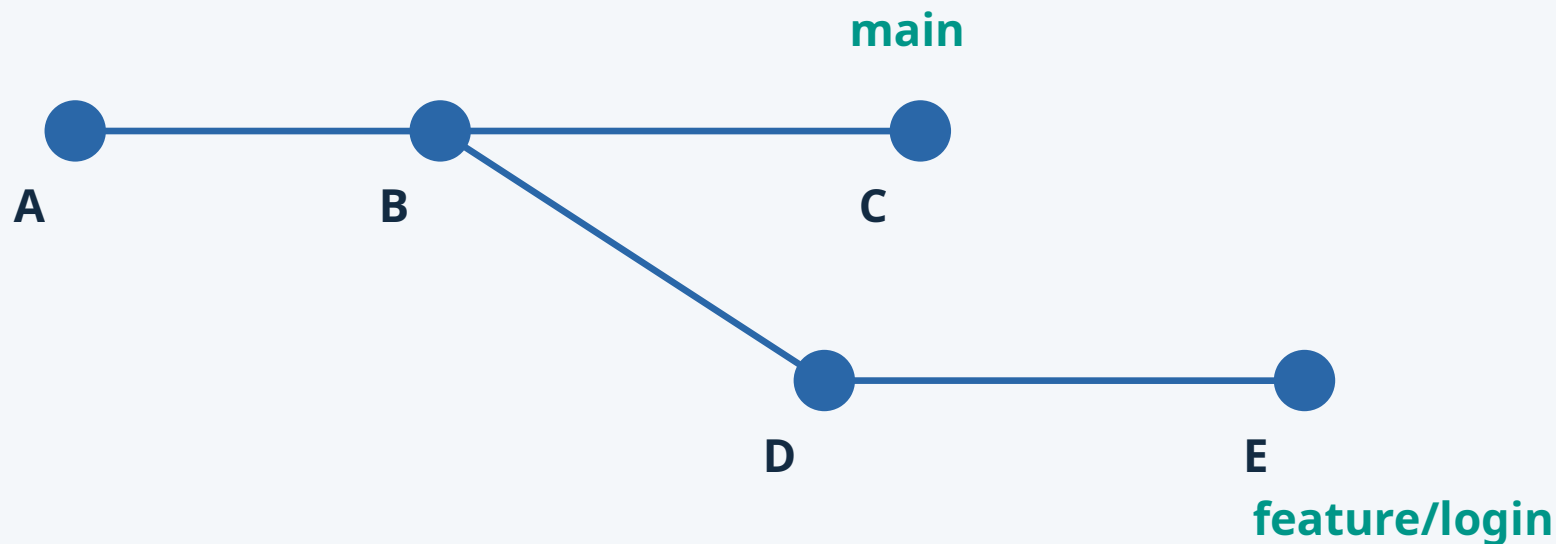


```
git switch main
git merge --ff-only feature/login
```

Fast-forward move a referência da branch. Não cria um commit M.

Merge de Três Vias — Histórias Divergentes

A main avançou até C; a feature avançou até E



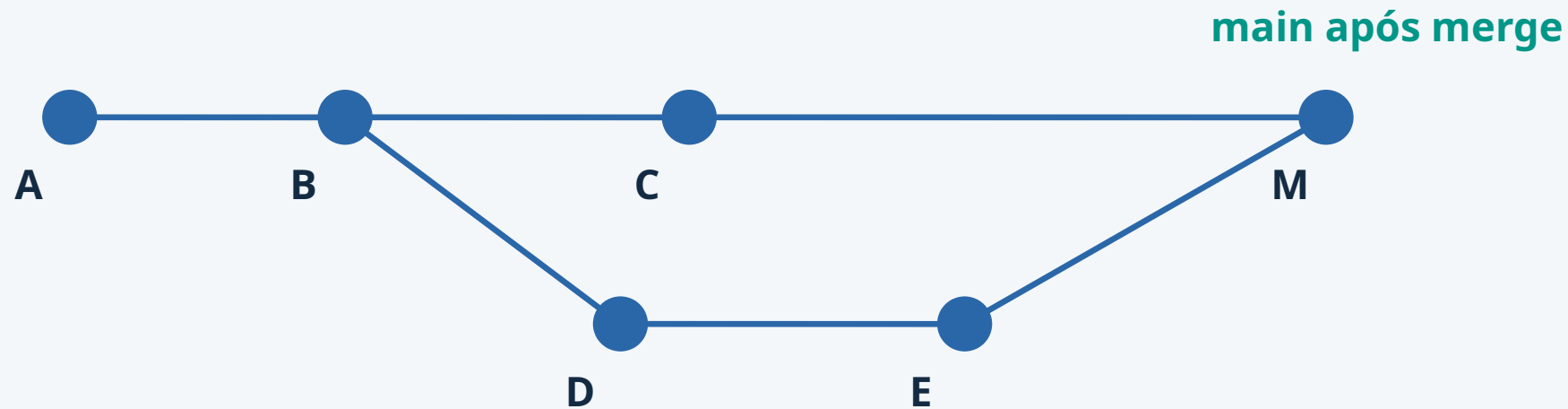
B → C: mudança na main

B → E: mudança na feature

As três versões são: base comum B, ponta C e ponta E.

Merge de Três Vias — Commit de Junção

Git combina as mudanças relativas à base comum



```
git switch main
git merge feature/login
```

M tem dois pais: C e E. As duas linhas continuam visíveis no histórico.

Três Vias Não Significa Conflito

A divergência do histórico e a incompatibilidade de conteúdo são coisas distintas

Integração automática

C altera uma instrução no README.
E adiciona testes em outro arquivo.
Git pode combinar sem intervenção.

Conflito textual

C define porta 8080.
E define porta 9090 na mesma linha.
A equipe decide o valor correto.

Um merge pode ser automático e ainda introduzir um erro de comportamento.

Fast-forward x Commit de Merge

Compare o efeito no grafo, não apenas o comando

Fast-forward

- Exige relação de ancestralidade
- Não cria commit de junção
- Histórico segue a mesma linha

Commit de merge

- Registra uma integração
- Tem dois pais no caso comum
- Mantém as linhas das branches

A escolha também expressa a política de histórico da equipe.

O Que Mudam `--ff-only` e `--no-ff`?

Mesmo cenário: main em B e feature em D, sem divergência

`--ff-only`

A — B — C — D

main avança até D.

Se main tivesse outro caminho, o comando falharia.

`--no-ff`

Preserva C e D e cria M.

M registra explicitamente a integração da feature.

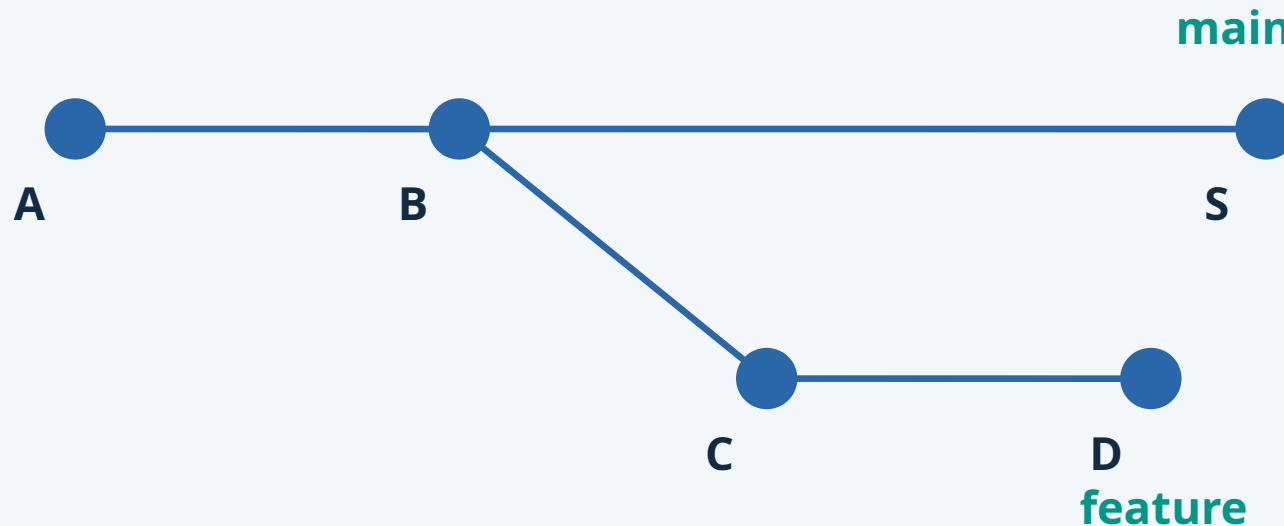
Útil para destacar a fronteira da mudança.

```
git merge --ff-only feature/login
git merge --no-ff feature/login
```

`--no-ff` força uma junção quando há algo a integrar; `--ff-only` recusa divergência.

Squash: Uma Mudança, Um Novo Commit

Não confunda condensar mudanças com preservar a junção das branches



```
git merge --squash feature/login
git commit -m "feat: adiciona login"
```

O commit S não tem a ponta da feature como segundo pai. Merge normal: junta as histórias da branch – Squash merge: Junta mudanças mas não conecta as histórias

Qual Histórico Você Espera Ver?

Exemplos para discutir em sala, sem executar comandos

Cenário 1

main está em B; feature em D na mesma linha.

Esperado: fast-forward é possível.

Cenário 2

main tem C; feature tem D e E a partir de B.

Esperado: merge de três vias.

Cenário 3

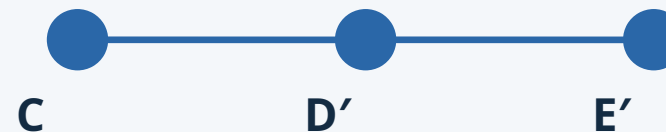
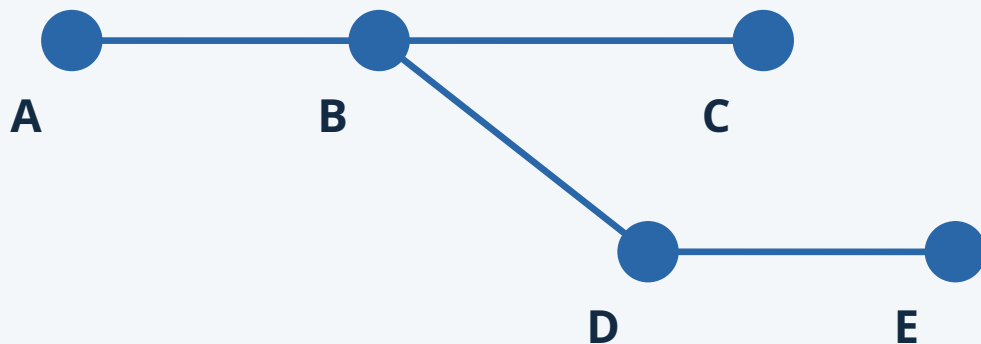
A equipe quer uma única mudança final para vários commits.

Esperado: avaliar squash e rastreabilidade.

Identifique ancestralidade, forma do histórico e evidência necessária.

Rebase: Mudando a Base da Minha História

Reaplicar commits locais sobre uma base mais recente



```
git switch feature/login
git fetch origin
git rebase origin/main
```

D e E viram D' e E': o conteúdo pode ser equivalente, mas os hashes mudam.

Merge x Rebase

Escolha consciente de como integrar

Merge

- Preserva commits existentes
- Pode criar commit de junção
- Adequado a branches compartilhadas

Rebase

- Reaplica commits em outra base
- Produz novos hashes
- Exige cuidado com histórico publicado

Rebase reescreve commits: evite aplicá-lo a histórico compartilhado.

Quando Duas Pessoas Alteram a Mesma Região

Conflito é uma decisão de integração que precisa de contexto

```
<<<<<< HEAD  
Porta da aplicação: 3000  
=====  
Porta da aplicação: 8080  
>>>>>> feature/config
```

1. Leia os dois lados e confirme o requisito.
2. Edite o resultado final e remova os marcadores.
3. Teste o comportamento antes de continuar.

Git aponta a incompatibilidade; a equipe decide o comportamento correto.

Resolver ou Cancelar um Conflito

Depois de editar e testar, registre a decisão

Merge

`git status`

`git add README.md`

`git commit`

Cancelar: `git merge --abort`

Rebase

`git status`

`git add README.md`

`git rebase --continue`

Cancelar: `git rebase --abort`

Não remova marcadores sem entender qual resultado a aplicação precisa.

Nem Tudo Deve Ir Para o Git

.gitignore evita incluir arquivos não rastreados

```
node_modules/  
.env  
dist/  
coverage/  
*.log
```

Não versionar dependências instaladas, builds locais e logs.

Nunca publicar tokens, senhas, chaves privadas ou credenciais.

.gitignore não deixa de rastrear arquivos já versionados.

Se um segredo foi publicado, revogue-o: apagar o arquivo não apaga o histórico.

Mapa Mental dos Comandos Git

Referência de consulta: agrupe por intenção

Criar / obter

git init • git clone

Inspecionar

git status • git diff • git log

Branches

git branch • git switch

Registrar

git add • git commit

Remoto

git remote • fetch • pull •
push

Integrar / marcar

git merge • git rebase • git
tag

Primeiro identifique a intenção; depois escolha o comando.

Modelos de Branching

A disciplina adota GitHub Flow

Trunk-based

Integração muito frequente
Branches muito curtas
Automação forte; flags quando necessárias

GitHub Flow

Branch por mudança
PR, revisão e checks
Merge na main estável

GitFlow

main e develop
feature, release e hotfix
Maior coordenação

GitHub Flow oferece um fluxo simples e rastreável para o tamanho das equipes.

Fluxo Git Recomendado

Após a inicialização: main estável e mudanças em branches de trabalho



Trabalho

feature/login
feature/readme
feature/testes

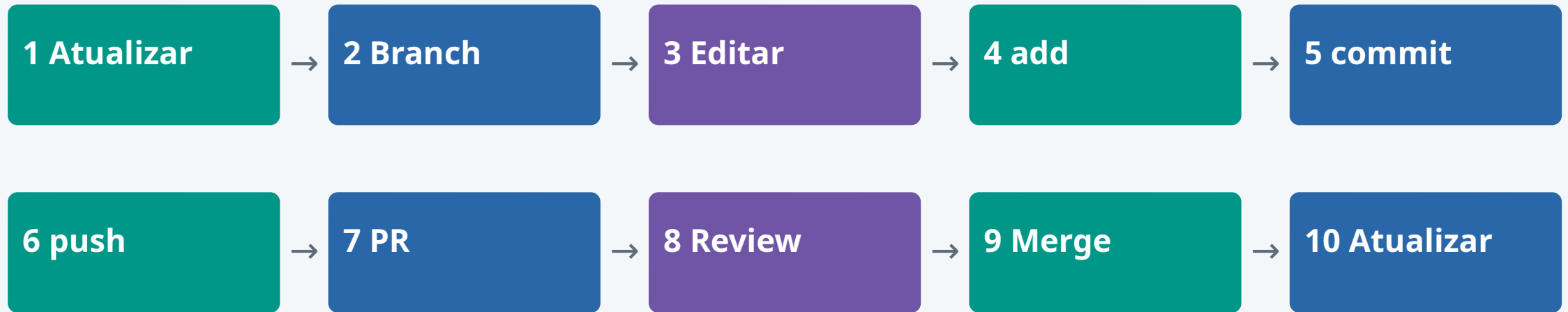
Integração

Após o commit inicial: usar branch e PR
Revisar antes de integrar
Atualizar a cópia local depois

Mudança relevante → PR com contexto, testes e risco → aprovação → merge.

Da Tarefa ao Merge

O ciclo completo de uma mudança



```
git switch main
git pull --ff-only origin main
git switch -c feature/login
```

O push publica a branch; o PR organiza sua revisão e integração.

Commits Que Contam a História

Tipo + descrição da intenção da mudança

Exemplos claros

feat: adiciona autenticação JWT

fix: corrige validação do CPF

docs: explica execução local

Outras intenções

test: cobre entrada inválida

refactor: extrai validação

Um propósito por commit

Evite mensagens como “ajustes”, “fix”, “teste” e “agora vai”.

Anatomia de um Bom Pull Request

Dê ao revisor contexto e evidência

Contexto

Problema e objetivo
Solução proposta
Arquivos e áreas afetadas

Validação

Comandos executados
Resultados dos testes
Como reproduzir

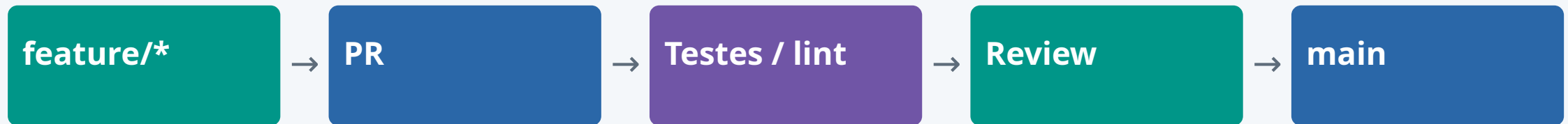
Risco

Regressões possíveis
Limitações conhecidas
Pontos de atenção

PR pequeno é mais fácil de revisar, compreender e reverter.

Protegendo a Branch Principal

Após publicar a base inicial: branch protection e quality gates



Regras possíveis

- Exigir PR e aprovação
- Exigir status checks
- Impedir merge com checks falhando

Integridade do histórico

- Restringir force push
- Impedir exclusão da main
- Definir quem pode contornar regras

Proteções transformam o acordo da equipe em regras verificáveis.

Definition of Done

Concluído significa verificável

Código

Revisado por outra pessoa
Testes relevantes passando
Lint e build funcionando

Entrega

Sem segredo exposto
README atualizado
Executa após clone limpo

Evidência

PR com contexto e risco
Validação reproduzível
Sugestões de IA verificadas

O checklist deve ser adequado à aplicação e explícito para todos.

Revisão de PR com IA

Preparar contexto → pedir análise → validar

Contexto

Objetivo e diff
Stack e critérios de aceite
Testes existentes; sem segredos

Análise

Bugs e regressões
Segurança e tratamento de erros
Testes faltantes e legibilidade

Validação humana

Conferir a sugestão
Executar teste, lint e build
Rejeitar mudanças incorretas

Toda saída de IA é hipótese até ser validada por teste, execução ou revisão humana.

Ferramentas de IA para Revisão

Exemplos para explorar; recursos e acesso variam por produto

Ferramentas

GitHub Copilot
CodeRabbit
Qodo

Outras opções

Greptile
Graphite
Avaliar integração e
contexto

Na prática da turma

CodeRabbit no PR
Comparar diagnóstico com
teste
Registrar decisão da equipe

A IA pode ser o primeiro leitor do PR; a decisão continua humana.

Qualidade Além do Olho Humano

Cada camada encontra classes diferentes de problemas

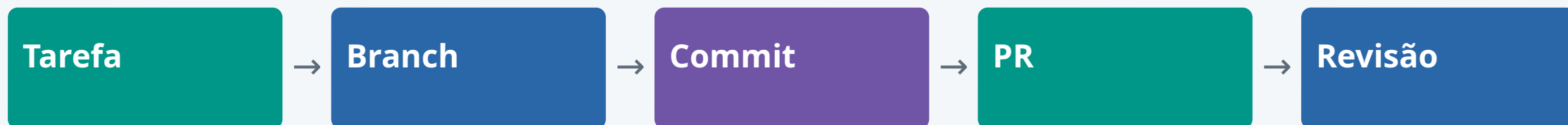


Estilo e convenções → padrões de risco → comportamento executável → hipóteses sobre a mudança → contexto e decisão de negócio.

Nenhuma camada substitui todas as outras.

Exemplo Comentado — Da Mudança à Revisão

Apresenta uma mudança posterior à inicialização do projeto



O que observar

Qual problema a mudança resolve?
 Por que o trabalho está isolado?
 O diff é pequeno e compreensível?

O que pedir

Teste que reproduza o comportamento
 Risco conhecido e contexto
 Decisão explícita de um revisor

Exemplo de evolução: compreender a revisão antes de aplicá-la às próximas mudanças.

Demonstração — CodeRabbit no PR

Use o PR preparado pelo professor para observar a revisão



Exemplo: 10% de desconto em 2000 centavos deve resultar em 1800.
O código retorna 1990, embora os testes iniciais passem.
Discuta o que a revisão encontrou e qual teste faltava.

A IA propõe hipóteses; testes e revisão humana sustentam a decisão.

Discussão Orientada — Aceitar ou Rejeitar?

Analise a revisão apresentada, sem criar branches ou commits

Analisar

Leia o contrato e o trecho
Identifique a entrada problemática
Compare esperado e observado

Decidir

A sugestão resolve o problema?
Que regressão ela pode introduzir?
Qual teste comprova a correção?

Registrar

Prompt e resposta apresentados
Sugestão aceita ou rejeitada
Justificativa e validação

Entregável da discussão: diagnóstico, evidência e justificativa da decisão.

Atividade 2 — Iniciar o Projeto da Equipe

Incremento do projeto integrador: do diagnóstico à primeira base executável



Um integrante cria a organização e o repositório do grupo.
Todos aceitam o convite e confirmam acesso.
A equipe constrói a base da aplicação e publica o primeiro commit.

Um único projeto evolui: E1 diagnóstico → E2 base publicada → E3 integração contínua.

Atividade 2 — Organização e Repositório

Etapa 1 • Um responsável prepara o espaço compartilhado

Organização

Definir nome da equipe
Um integrante cria a organização no GitHub
Usar contas pessoais individuais

Repositório

Criar o repo do projeto na organização
Definir nome e descrição
Criar vazio, sem README automático

Responsabilidade

Criador assume administração inicial
Registrar quem administra
Garantir acesso do professor à entrega

O repositório pertence à organização da equipe, não à conta pessoal do criador.

Atividade 2 — Convidar e Confirmar Acessos

Etapa 2 • Convite enviado ainda não significa participação ativa

Convidar



Aceitar



Dar acesso



Confirmar

Responsável

- Convidar todos os integrantes
- Conceder acesso de escrita ao repo
- Usar equipe do GitHub ou acesso individual

Cada integrante

- Aceitar com a própria conta
- Confirmar que enxerga o repositório
- Verificar que poderá contribuir

Pertencer à organização não garante, sozinho, permissão de escrita no repositório.

Atividade 2 — Construir o Código-base

Etapa 3 • Implementar o menor comportamento executável do projeto

Aplicação mínima

Stack já escolhida pela equipe

Uma funcionalidade ou endpoint simples

Execução local comprovada

Arquivos da base

Código-fonte e dependências

README com execução e teste

.gitignore adequado à stack

Validação

Executar antes de publicar

Incluir um teste simples

Registrar resultado esperado e obtido

Escolha livre da aplicação; o exemplo CodeRabbit não substitui o projeto do grupo.

Atividade 2 — Primeiro Commit na main

Etapa 4 • Exceção de inicialização: publicar a primeira base do projeto

```
git init -b main  
git status  
git add .  
git diff --staged
```

```
git commit -m "chore: inicia base do projeto"  
git remote add origin <url-do-repositorio-vazio>  
git push -u origin main
```

Antes do commit: revisar arquivos selecionados e excluir segredos e artefatos locais.

Somente esta inicialização vai direto à main. As próximas mudanças seguem por branch e PR.

Atividade 2 — Validar a Base Compartilhada

Etapa 5 • Outro integrante confirma a execução em um clone limpo

Segundo integrante

Obter o projeto em pasta nova
Seguir apenas o README
Executar a aplicação e o teste

Equipe

Corrigir instruções insuficientes
Registrar evidência de execução
Relacionar código ao diagnóstico do E1

Próximas mudanças

Branch de trabalho → PR
Revisão e validação
Proteções da main após inicialização

Publicar código não basta: a base precisa ser reproduzível por outra pessoa.

Atividade 2 — Critérios de Aceite

Entrega ao final do encontro: link do repositório do projeto integrador

Equipe organizada

Organização criada
Repositório do projeto criado
Convites aceitos e escrita confirmada

Base publicada

Código-base na main
Primeiro commit identificável
README e .gitignore incluídos

Execução comprovada

Aplicação e teste executam
Outro integrante valida o clone
Evidência e uso de IA registrados

Não é necessário PR ou tag nesta inicialização; eles serão usados na evolução do projeto.

Prompts Úteis — Revisão e Qualidade

Forneça contexto suficiente e peça resultados verificáveis

Revisar diff

“Com estes critérios de aceite, encontre bugs e regressões. Cite a linha e uma entrada de reprodução.”

Sugerir testes

“Quais casos faltam para os limites e erros desta função? Explique o que cada teste comprova.”

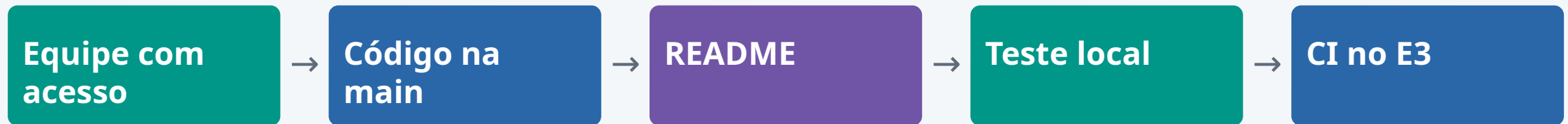
Avaliar risco

“Que hipótese sua depende de contexto ausente? Como validá-la antes de alterar o código?”

Registre prompt, resposta, decisão e evidência de validação.

Depois do Encontro 2

O mesmo repositório recebe o pipeline no Encontro 3



Levar pronto: link do repo, código-base, comandos de execução e teste.

Próxima mudança: adicionar o pipeline em branch e abrir PR.

A CI automatiza a validação que a equipe já executou localmente.

E2: execução local reproduzível → E3: execução automatizada e status check.



Encontro 3

12/09/2026 | Integração contínua, testes e IA para automação

Integração Contínua, Testes e IA para Automação

12/09/2026 • Feedback rápido para reduzir risco de integração



Objetivo: transformar a validação local em validação automática, rastreável e reproduzível a cada PR/push.

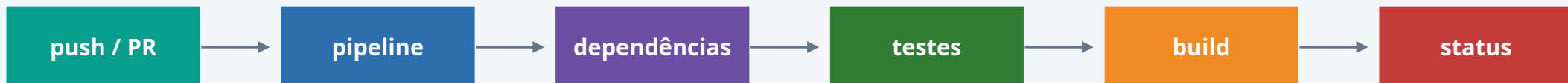
Entender → montar → quebrar → corrigir → evidenciar

CI em uma frase

Integração Contínua é validação automática e frequente do código

CI responde uma pergunta simples:

“Essa mudança ainda deixa o projeto funcionando?”



Antes da CI

A validação dependia da máquina e disciplina de cada pessoa.

Resultado: “funciona na minha máquina”.

Com CI

A validação roda em ambiente padronizado, vinculada ao commit e ao PR.

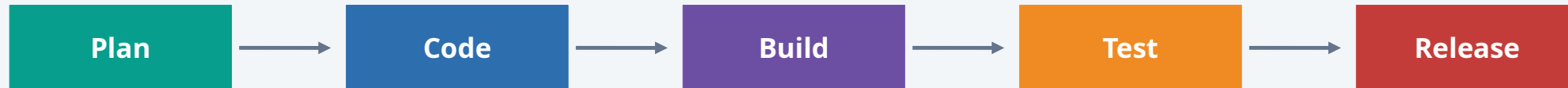
Resultado: evidência objetiva.

Regra da aula

Pipeline vermelho é feedback; pipeline ignorado é risco.

Onde a CI entra no fluxo DevOps?

Ela fecha o caminho entre Code, Build e Test com feedback rápido



Sem CI: o erro pode ser descoberto tarde, depois que várias mudanças já foram misturadas.



Com CI: cada alteração recebe um sinal rápido, reproduzível e ligado ao commit.

O que a CI protege

- Integração entre mudanças
- Comandos de build/teste
- Confiança no Pull Request
- Evidência para revisão e release

O que a CI não garante

- Que a regra de negócio está correta
- Que há teste suficiente
- Que o design está bom
- Que não existe risco operacional

Mensagem central

CI não substitui engenharia. Ela acelera feedback e aumenta rastreabilidade.

Validação local x Validação no pipeline

As duas são necessárias, mas respondem perguntas diferentes

Local

Pergunta: “na minha máquina está funcionando?”

Comandos típicos:

- npm test
- npm run build
- execução manual

Risco: depende do ambiente local.

CI

Pergunta: “funciona em ambiente limpo e rastreável?”

Comandos típicos:

- npm ci
- npm test
- npm run build

Vantagem: gera log e status vinculados ao commit.

Produção

Pergunta: “o usuário real está bem atendido?”

Sinais:

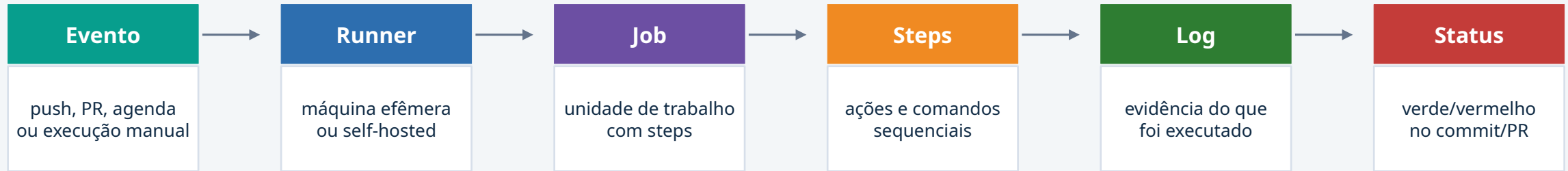
- logs
- métricas
- health checks
- incidentes

Entrará com mais força nos encontros 4, 5 e 6.

Boa prática: rode localmente antes de abrir PR; deixe a CI confirmar em ambiente padronizado.

Modelo mental de um pipeline

Um pipeline é uma sequência rastreável de verificações



```
name: ci
on: [push, pull_request]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
      - run: npm run build
```

Como explicar em sala

O YAML não é “código mágico”. Ele declara quando executar, onde executar e quais comandos comprovam que o projeto continua íntegro.

A pergunta que guia o desenho

Que validação mínima precisa falhar se alguém quebrar o projeto?

Anatomia de um workflow GitHub Actions

Ler o YAML de cima para baixo ajuda a entender o pipeline

```
name: ci

on:
  pull_request:
  push:
    branches: [main]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test
      - run: npm run build
```

1. name

Nome do workflow exibido na aba Actions.

2. on

Define os gatilhos: PR, push, agenda ou manual.

3. jobs

Agrupa as unidades de execução do pipeline.

4. runs-on

Escolhe o runner: exemplo ubuntu-latest.

5. steps

Executa ações e comandos em sequência.

6. run

Comandos reais do projeto: instalar, testar e gerar build.

Trigger: quando o pipeline deve rodar?

O gatilho errado gera desperdício ou deixa risco passar

pull_request

Roda antes do merge.

Uso na disciplina: validar mudanças propostas antes de entrar na main.

push na main

Roda depois que a main recebeu mudança.

Uso: gerar evidência da versão integrada.

workflow_dispatch

Roda manualmente.

Uso: reexecutar uma validação ou demonstrar o pipeline em sala.

```
on:  
  pull_request:  
  push:  
    branches: [main]  
  workflow_dispatch:
```

Pergunta de arquitetura: qual evento entrega feedback cedo sem gastar execução à toa?

Para o projeto integrador: comece com pull_request + push na main.

Runner, job e step sem complicar

Três conceitos que aparecem em quase todo pipeline

Runner

É a máquina que executa o job.

Exemplo: ubuntu-latest.

Pode ser hospedado pelo GitHub ou self-hosted.

Job

É uma unidade de trabalho.

Pode rodar build, testes, análise de qualidade ou publicação de artefato.

Step

É uma ação ou comando dentro do job.

Exemplo: checkout, setup-node, npm ci, npm test.

```
jobs:  
  build-test:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - run: npm ci  
      - run: npm test
```

Leia assim: “No job build-test, rode em Ubuntu e execute estes passos.”

Os comandos Node.js dentro da CI

O pipeline executa comandos que já devem existir no projeto

npm ci

Instala dependências exatamente conforme package-lock.json.

Ideal para CI porque não altera o lockfile e falha se houver inconsistência.

npm test

Executa o script de testes configurado no package.json.

Exemplo: Jest, Vitest, Mocha ou outro runner.

npm run build

Executa o script build.

Serve para provar que a aplicação compila, empacota ou gera artefato.

```
{
  "scripts": {
    "test": "vitest run",
    "build": "vite build"
  }
}
```

Se o script não existe no package.json, o pipeline falha. Isso é bom: o erro aparece cedo.

Por que usar npm ci no pipeline?

CI precisa de instalação limpa e reproduzível

```
package.json  
package-lock.json  
|  
v  
npm ci  
|  
v  
node_modules limpo
```

Comportamento importante

- exige package-lock.json
- remove node_modules existente
- não altera package.json
- não altera package-lock.json
- falha se lock e package divergem

Por que isso importa?

Na CI, você quer descobrir se o projeto funciona a partir de um clone limpo, não se ele funciona por acidente na máquina de alguém.

Frase para memorizar: npm install é para evoluir dependências; npm ci é para reproduzir dependências.

Status check: o pipeline vira regra de qualidade

O resultado da CI aparece no commit e no Pull Request



Sem checks obrigatórios

O time pode fazer merge mesmo quando testes falham ou nem foram executados.

Com checks obrigatórios

A main só recebe mudanças se o pipeline passar. A regra deixa de ser verbal e vira automatizada.

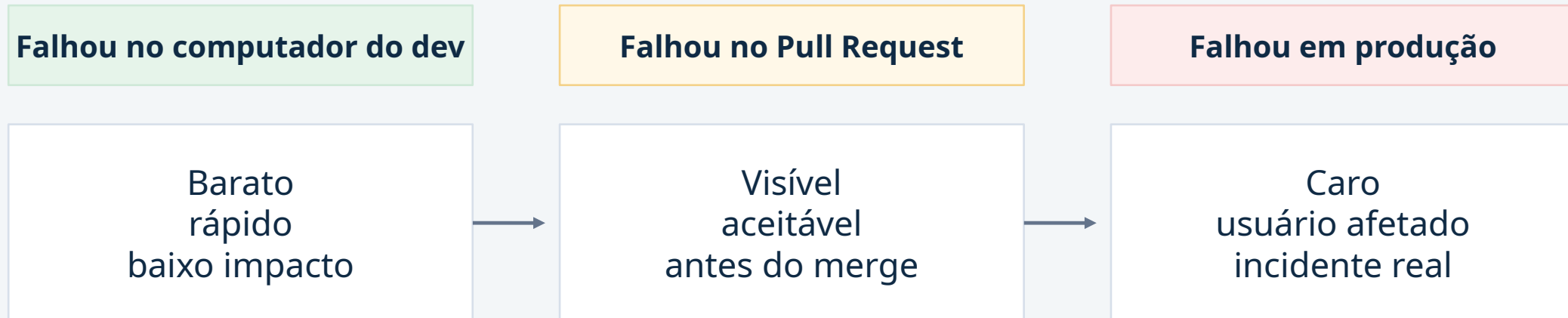
Conexão com Encontro 2

Branch protection + PR + CI transforma colaboração em fluxo controlado.

Na disciplina: a CI do Encontro 3 prepara a main para receber quality gates nos próximos incrementos.

Pipeline vermelho é feedback

Falhar cedo é uma das ideias mais importantes de DevOps



Objetivo da CI: empurrar a descoberta de falhas para a esquerda, antes da entrega.

Pirâmide de testes com exemplos

Escolha testes que dão feedback rápido e comprovam comportamento



Unitário

Exemplo: calcular desconto, validar CPF, regra de frete.

Feedback: segundos.

Integração

Exemplo: API grava pedido no banco ou consulta serviço fake.

Feedback: mais lento.

Smoke

Exemplo: aplicação sobe e /health retorna 200.

Feedback: prova mínima de vida.

Decisão

No projeto de 24h, não tente cobrir tudo. Prove o fluxo crítico e gere evidência.

Exemplo de teste unitário

Testa uma função ou regra isolada, sem depender de banco ou rede

```
export function applyDiscount(total, percent) {  
  return total - total * percent / 100;  
}  
  
test("10% de 2000 centavos = 1800", () => {  
  expect(applyDiscount(2000, 10)).toBe(1800);  
});
```

O que esse teste comprova

- entrada conhecida
- saída esperada
- regra de negócio isolada
- execução rápida

Por que cabe na CI

Teste unitário é barato. Ele deve rodar em todo Pull Request porque dá feedback rápido.

Pergunta para a turma: que entrada quebraria essa função? E se percent vier negativo?

Exemplo de teste de integração

Testa colaboração entre partes da aplicação



```
POST /orders
{ "customerId": 10, "items": [...] }

expect(response.status).toBe(201)
expect(order).toExistInDatabase()
```

O que comprova

A rota, a regra e a persistência trabalham juntas.

Cuidado

Pode ficar lento ou frágil se depender de rede real ou dados compartilhados.

Na disciplina: use integração quando isso provar o fluxo principal do projeto.

Exemplo de smoke test

Prova mínima: a aplicação sobe e responde

```
# depois do build ou container subir  
curl -f http://localhost:3000/health  
  
# esperado:  
{ "status": "ok" }
```

Smoke test não prova todos os comportamentos. Ele prova que o sistema está vivo o suficiente para continuar a validação.

Rápido

Poucos

Críticos

Úteis para deploy

Exemplo no projeto final: endpoint /health + comando documentado no README.

Teste ruim x teste útil

Um pipeline verde só tem valor se os testes realmente protegem comportamento

Teste ruim

```
test("existe", () => {  
  expect(service).toBeDefined();  
});
```

Problema: quase não prova comportamento real.

Teste útil

```
test("calcula desconto", () => {  
  expect(applyDiscount(2000, 10))  
    .toBe(1800);  
});
```

Valor: protege uma regra esperada.

Pergunte sempre: “Que bug esse teste conseguiria encontrar?”

Testes frágeis: quando o pipeline perde credibilidade

Flaky test falha de forma intermitente e destrói confiança na CI

Causas comuns

- dependência de horário
- ordem de execução
- rede real
- dados compartilhados
- condições de corrida

Como tratar

- isolar estado
- usar mocks/fakes
- limpar dados entre testes
- remover dependência de sorte
- investigar como bug real

O que não fazer

- reexecutar infinitamente
- ignorar falha
- marcar como "normal"
- esconder erro para passar entrega

Discussão: um teste que falha 1 vez a cada 20 execuções deve bloquear a main?

Métricas de CI que contam a verdade

Medir o pipeline ajuda a melhorar o fluxo de entrega

Tempo de build

Quanto tempo o time espera pelo feedback?

Sinal ruim: builds longos demais fazem a equipe evitar integração frequente.

Taxa de falha

Quantos pipelines quebram?

Sinal útil quando aponta regressão, falta de teste local ou instabilidade do projeto.

Flakiness

Quantas falhas são intermitentes?

Sinal crítico: destrói confiança e leva o time a ignorar o pipeline.

Cobertura

Útil como sinal de risco, perigosa como meta isolada.

100% de cobertura não garante ausência de bugs.

Taxa de sucesso na main

A main deve estar saudável.

Pipeline vermelho na main é prioridade de correção.

Como ler um log de pipeline

O log é a evidência técnica da falha

```
Run npm test
FAIL src/discount.test.ts
Expected: 1800
Received: 1990

at discount.test.ts:8:32
Error: Process completed with exit code 1.
```

1. Onde falhou?

Identifique job, step e comando.

Exemplo: step npm test.

2. Qual evidência?

Compare esperado e obtido.

Exemplo: esperado 1800, recebido 1990.

3. Qual próxima ação?

Reproduzir localmente, escrever teste mínimo e corrigir causa.

Não corrija “no escuro”: primeiro transforme log em hipótese verificável.

IA para investigar falhas de CI

Use IA para levantar hipóteses, não para pular validação

Contexto seguro

Cole apenas trechos relevantes do log.

Remova tokens, URLs internas sensíveis e dados pessoais.

Pedido correto

Peça hipóteses em ordem de probabilidade e comandos para validar antes de alterar código.

Validação humana

Teste a hipótese, rode o pipeline novamente e registre o que foi aceito ou rejeitado.

Prompt exemplo:

Este job de CI falhou. Explique a provável causa, liste hipóteses e sugira validações antes de alterar código.

Log: <trecho sem segredos>

Regra: toda resposta da IA é hipótese até ser confirmada por teste, build ou revisão.

IA para gerar testes: onde ajuda e onde engana

A IA acelera ideias; a equipe decide cobertura e relevância

Ajuda bem em

- casos de borda
- entradas inválidas
- nomes de testes
- estrutura inicial
- cenários esquecidos

Pode enganar em

- API inexistente
- mocks irreais
- teste que replica bug
- cobertura sem valor
- falsa segurança

Checklist humano

- o teste falha antes da correção?
- prova comportamento real?
- roda no CI?
- não depende de sorte?

Prompt:

Com base nesta função e nos critérios de aceite, sugira testes unitários e casos de borda. Não altere a API pública sem justificar.

A IA gera candidatos. A qualidade do teste é responsabilidade do time.

Demonstração sugerida: quebrar, detectar e corrigir

Uma boa aula de CI precisa mostrar o vermelho e o verde



Expected: 1800
Received: 1990

Causa provável:
percentual aplicado como 0.5% em vez de 10%

Perguntas para a turma

- Qual teste faltava?
- Onde o log aponta a causa?
- A sugestão da IA é suficiente?
- Que evidência anexar no PR?

Objetivo da demo: CI não é teoria. É feedback executável ligado a uma mudança.

Atividade 3 — Pipeline de CI

Provar, com log, que o pipeline protege a entrega

Criar

Arquivo versionado:
.github/workflows/ci.yml

O workflow deve rodar em PR e push na main.

Validar

Executar no pipeline:

- instalação
- teste real
- build

Falha deve bloquear a entrega.

Evidenciar

Entregar:

- print ou link do pipeline verde
- um log de falha explicado
- correção aplicada
- registro de IA, se usada

Critério forte: deve existir pelo menos um teste que falha se o comportamento principal for quebrado.

```
npm ci
npm test
npm run build
```

Checklist de evidências do Encontro 3

O que deve ficar claro para avaliação e continuidade

Repositório

- workflow versionado
- README com comandos
- scripts definidos
- dependências travadas

Pipeline

- executa em PR/push
- logs disponíveis
- status verde
- falha analisada

Teste

- pelo menos um teste real
- prova comportamento
- não é trivial
- roda no CI

IA

- prompt sem segredo
- hipótese registrada
- decisão humana
- validação por execução

Próximo encontro

Levar porta, variáveis, dependências externas e pendências do CI para containerizar a aplicação.

Depois do Encontro 3

A validação automatizada vira base para containers e Compose



Tarefa para o Encontro 4

- corrigir pendências do CI
- documentar build/teste no README
- confirmar porta da aplicação
- listar variáveis obrigatórias
- mapear dependências externas

Mensagem de fechamento

CI é o primeiro ambiente confiável fora da máquina do desenvolvedor. Containers levam essa reprodutibilidade para execução.

E3: execução automatizada → E4: execução reproduzível



Encontro 4

24/09/2026 | Containers, integração e troubleshooting

Encontro 4 - Containers e Compose

Ambiente reproduzível para desenvolvimento, validação e diagnóstico assistido

Dockerfile

- Imagem mínima e reproduzível
- Camadas e cache
- Variáveis de ambiente
- Execução local previsível
- IA para explicar erros de build sem expor segredos

Compose

- Serviços, redes e volumes
- Banco e dependências
- Health checks
- Logs de serviços

Laboratório

- Containerizar aplicação
- Subir stack local
- Validar integração API + banco
- Usar IA para diagnosticar falha de container
- Documentar comandos

Saída do encontro

- Aplicação executável por comando padronizado, com instruções, evidências de execução e correções explicadas.

Containers Explicados Sem Excesso

O essencial para operar e diagnosticar

Imagem

- Pacote versionável da aplicação.
- Criada por Dockerfile.
- Deve ser reproduzível, pequena e sem segredos.

Container

- Processo isolado criado a partir da imagem.
- Tem logs, portas, variáveis e ciclo de vida.
- Pode morrer: por isso health check e logs importam.

Compose

- Orquestra múltiplos serviços localmente.
- Padroniza API, banco, rede e volumes.
- Ótimo para laboratório e validação integrada.

Por Que Containers?

O problema que containers resolvem de verdade

O problema

- Diferenças de SO, versão de linguagem e dependências geram bugs que só aparecem em produção.

VM vs container

- VM virtualiza o hardware inteiro: pesada, minutos para subir.
- Container compartilha o kernel do host: leve, segundos para subir.

O que o container garante

- O mesmo ambiente do laptop do dev até a produção.
- Elimina uma classe inteira de bugs "ambiente-dependentes".

Dockerfile - Roteiro de Explicação

Use como base e adapte para a stack escolhida

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 3000

CMD ["npm", "start"]
```

O que comentar

- Base image, diretório de trabalho, cópia de dependências, cache, variáveis, porta e comando final.

Cuidados

- Não copiar .env, node_modules, chaves ou arquivos desnecessários.
- Usar .dockerignore e imagem base adequada.

Validação

- docker build deve funcionar do zero.
- docker run deve expor porta e responder endpoint simples.

Boas Práticas de Dockerfile: Camadas e Cache

Imagens mais rápidas de construir e mais leves

Como o cache funciona

- Cada instrução vira uma camada.
- Docker reaproveita camadas que não mudaram.
- Por isso: copiar package.json antes de copiar todo o código.

Multi-stage build

- Uma etapa compila o projeto.
- Outra etapa final só carrega o artefato pronto.
- Imagem final menor, sem ferramentas de build sobrando.

Escolha de imagem base

- alpine: mínima, pode faltar biblioteca do sistema.
- slim: equilíbrio entre tamanho e compatibilidade.
- full: mais compatível, porém mais pesada.

Compose - Roteiro de Laboratório

Padronizar execução local com serviços integrados

```
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://app:app@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: app
      POSTGRES_DB: app
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 5
```

Como explicar

- Cada serviço tem imagem/build, portas, variáveis e dependências.
- O nome do serviço vira hostname na rede do Compose.

Comandos em sala

- docker compose up --build
- docker compose logs -f app
- docker compose down -v

Evidência

- Print/log da aplicação subindo.
- Endpoint funcionando.
- README com comando único de execução.

Redes e Volumes no Docker

Dois conceitos que resolvem a maioria das dúvidas de Compose

Rede

- Containers no mesmo Compose se enxergam pelo nome do serviço, como hostname.
- Portas expostas ligam o container ao host.

Volumes nomeados

- Dados sobrevivem à recriação do container.
- Essencial para banco de dados e qualquer estado persistente.

Bind mounts

- Mapeiam uma pasta do host para dentro do container.
- Úteis em desenvolvimento para hot-reload de código.

Troubleshooting de Containers

Uma ordem para não perder tempo reescrevendo código à toa

Comandos de investigação

- `docker ps` — o que está rodando?
- `docker logs <id>` — o que o processo disse?
- `docker exec -it <id> sh` — inspecionar por dentro.
- `docker inspect <id>` — configuração completa.

Falhas comuns

- Porta não exposta ou não mapeada.
- Variável de ambiente faltando.
- Dependência (banco) ainda não está pronta.
- Permissão de arquivo incorreta.

Ordem de diagnóstico

- O container subiu? Os logs mostram erro? A variável está correta? A dependência está saudável?
- Siga essa ordem antes de reescrever código.

Atividade 4 - Containerização

Sair do “funciona na minha máquina” com evidência

Perguntas guia

- A aplicação sobe com um único comando, a partir de um clone limpo?
- Que variável de ambiente é obrigatória?
- Como o grupo diagnosticou a primeira falha de container?
- O que a IA sugeriu que precisou ser corrigido?

Evidências

- Dockerfile e compose.yaml versionados.
- Aplicação respondendo após docker compose up.
- Print ou log da stack subindo com sucesso.
- Uma falha de container diagnosticada e resolvida.

Entrega

- Dockerfile, compose.yaml e README atualizados no repositório.
- Evidência de execução anexada.

Conexão com avaliação

- Compõe os 20% de Containers/Compose da rubrica final.

Prompts Úteis - Containers

Use IA para diagnosticar, não para colar segredos



Erro de build

- Este build Docker falhou. Liste causas prováveis e comandos para validar antes de alterar o Dockerfile.
- Dockerfile: <trecho>. Log: <trecho sem segredos>.

Compose

- Analise este compose.yaml e aponte problemas de rede, portas, variáveis, depends_on, volumes e health checks.
- Arquivo: <trecho>.

README

- Gere instruções curtas para executar este projeto com Docker Compose: pré-requisitos, comando de subida, teste de saúde e limpeza.
- Contexto: <stack>.

Depois do Encontro 4

O que levar pronto para o Encontro 5: CD e governança

Tarefa

- Integrar build de imagem ao pipeline, se possível.
- Preparar uma versão candidata para release.
- Levar para o Encontro 5: critérios de promoção e rollback.



Encontro 5

25/09/2026 | CD, configuração, segurança e governança de IA

Encontro 5 - Entrega e Governança

Automação com controle, segurança, rollback e governança de IA

Entrega contínua

- Promoção de artefatos
- Ambientes e aprovações
- Release notes
- Rollback e feature flags

Configuração e IaC

- Configuração declarativa
- Noções de IaC
- GitOps como evolução
- Padronização e rastreabilidade

Segurança e IA

- Segredos fora do código e dos prompts
- Permissões mínimas
- Scan de dependências/imagens
- Gates de segurança
- Validar alucinação, vazamento e prompt injection

Saída do encontro

- Plano de release com critérios de promoção, rollback, segurança mínima, rastreabilidade e regras de uso responsável de IA.

CD na Prática

O foco é controlar a mudança

Entrega contínua

- Artefato gerado no CI é promovido.
- Ambiente recebe versão rastreável.
- Release notes explicam mudança e risco.

Controle

- Aprovação quando houver risco.
- Variáveis e segredos fora do repositório.
- Permissões mínimas no pipeline.

Rollback

- Voltar para versão anterior.
- Reverter feature flag.
- Restaurar configuração conhecida.
- Ter plano documentado antes do problema.

Estratégias de Deploy

Comparar abordagens antes de escolher uma

Rolling deploy



- Substitui instâncias antigas pelas novas aos poucos.
- Simples, mas o rollback não é instantâneo.

Blue-green



- Duas versões completas rodando; a troca de tráfego é instantânea.
- Rollback rápido, porém custa recursos em dobro.

Canary



- Libera para uma fatia pequena de usuários primeiro.
- Expande se as métricas estiverem saudáveis — risco pequeno em caso de erro.

 versão antiga

 versão nova

Feature Flags: Separando Deploy de Release

Colocar código em produção não é o mesmo que ativá-lo

O conceito

- Deployar o código não significa ativar a funcionalidade para todo mundo.
- A flag decide quem vê o quê e quando.

Usos práticos

- Lançamento gradual para grupos de usuários.
- Teste A/B entre variações.
- Desligar rapidamente uma funcionalidade com bug, sem reverter o deploy inteiro.

Cuidado

- Flags esquecidas viram dívida técnica.
- Toda flag precisa ter dono e prazo previsto de remoção.

Configuração, Segredos e IaC

Conceitos que sustentam a prática de entrega

Configuração

- Valores que mudam por ambiente.
- Exemplos: URL, porta, modo debug, timeout.
- Não deve exigir alteração de código para trocar ambiente.

Segredos

- Senha, token, chave privada, connection string sensível.
- Nunca versionar ou colar em prompt.
- Usar secrets do provedor de CI/CD.

IaC/GitOps

- Infra declarada em arquivo e revisada por PR.
- GitOps: estado desejado versionado.
- Entra como noção e evolução, não como laboratório obrigatório.

DevSecOps Mínimo para a Disciplina

Gates simples que cabem no projeto integrador

Código

- Lint/análise estática.
- Testes mínimos em PR.
- Revisão de entrada de usuário e tratamento de erro.

Dependências/imagem

- Checar pacotes vulneráveis.
- Evitar imagem desatualizada ou grande demais.
- Não rodar container com privilégio desnecessário.

Pipeline

- Segredos mascarados.
- Permissão mínima do token.
- Logs sem dados sensíveis.
- Falha de segurança deve bloquear ou justificar exceção.

Correção de Vulnerabilidades com IA

De "encontrado" para "corrigido" — automaticamente

GitHub Copilot Autofix

- Usa CodeQL + LLM; cobre 90%+ dos tipos de alerta em JS/TS/Java/Python.
- Resolve 2/3 das vulnerabilidades com pouca edição — 3x mais rápido que manual.

Snyk Agent Fix

- Arquitetura agêntica (2026), usa base de 35 mil correções de especialistas como contexto.
- Se a correção falhar no re-scan, o erro volta pro modelo tentar de novo.

O que não muda

- A IA prioriza por risco e explorabilidade, não só severidade.
- Aceitar o fix ainda exige revisão humana e re-execução de testes.

Governança de IA em DevOps

Regras que entram na avaliação

Riscos a tratar

- Vazamento de segredos em prompts.
- Alucinação de comandos, APIs ou flags.
- Código inseguro gerado com confiança excessiva.
- Prompt injection em contexto de logs, issues ou documentação.
- Dependência intelectual: não saber defender o que foi entregue.

Controles esperados

- Mascaram dados sensíveis antes de usar IA.
- Pedir hipóteses e validações, não só resposta final.
- Rodar testes, build e revisão humana.
- Manter diff pequeno e rastreável.
- Registrar prompt, saída útil, decisão e validação.

Agentes Autônomos de DevOps

A fronteira mais recente do uso de IA — e por que a validação humana continua obrigatória

O que já existe

- Recebem uma issue, analisam o repositório, escrevem código, rodam teste e scan de segurança.
- Trabalham em background, de forma assíncrona, sem intervenção durante a execução.

Onde já é realidade

- GitHub Copilot coding agent (GA) e agentes equivalentes de outros provedores.
- Integrados a VS Code, JetBrains, Visual Studio e outros editores.

O limite que não muda

- Mesmo o agente mais autônomo para no Pull Request — não faz merge sozinho.
- Quanto mais autônoma a IA, mais crítica fica a revisão humana antes da produção.

Atividade 5 - Release e Rollback

Controlar a mudança, não só publicá-la

Perguntas guia

- O que precisa ser verdade para promover esta versão?
- Qual seria o primeiro sinal de que algo deu errado?
- Como o grupo voltaria para a versão anterior em produção?
- Que segredo não pode aparecer em log ou prompt?

Evidências

- Release notes preenchidas com o modelo fornecido.
- Plano de rollback documentado.
- Checklist de segurança básica revisado.
- Registro de uso de IA em decisão de entrega, quando houver.

Entrega

- Release notes e plano de rollback no repositório.
- Checklist de segurança preenchido.

Conexão com avaliação

- Alimenta os 15% de Entrega e Operação da rubrica final.

Release Notes e Rollback - Modelo

Artefato simples para o Encontro 5

Release v0.2.0

Objetivo:

- <qual problema esta versao resolve>

Mudancas:

- <mudanca 1>

- <mudanca 2>

Evidencias:

- CI: <link/print>

- Testes: <resumo>

- Container: <comando/print>

Riscos:

- <risco e mitigacao>

Plano de rollback

Quando acionar:

- <sinal de falha>

Como voltar:

1. Identificar versao anterior estavel
2. Reverter tag/imagem/configuracao
3. Validar endpoint /health
4. Comunicar impacto e decisao

Depois do Encontro 5

O que levar pronto para o Encontro 6: observabilidade e apresentação final

Tarefa

- Preparar observabilidade mínima.
- Simular uma falha ou incidente simples.
- Organizar apresentação final com evidências e decisões.



Encontro 6

26/09/2026 | Observabilidade, IA aplicada e projeto final

Encontro 6 - Operação e Evidências

Fechar o ciclo com dados, aprendizagem e análise assistida validada

Observabilidade

- Logs, métricas e traces
- Dashboards e alertas
- SLI/SLO
- IA para sumarizar logs e levantar hipóteses de causa
- Validação operacional de release

Evolução do pipeline

- Dados como base da automação
- IA aplicada a troubleshooting e documentação operacional
- AIOps como tendência, não substituto de decisão técnica
- Automação orientada por eventos
- Limites e próximos passos

Apresentações

- Demonstração do projeto
- Evidências do pipeline
- Diagnóstico, proposta e registro de IA
- Tradeoffs e próximos passos

Saída do encontro

- Entrega final avaliada por evidências: repositório, pipeline, container, release, observabilidade, registro crítico de IA e defesa técnica.

Observabilidade Sem Complicar

Usar dados para explicar se a entrega funcionou

Logs

- Eventos textuais: erro, início, fim, exceção.
- Devem permitir entender o que aconteceu.
- Evitar dados sensíveis em logs.

Métricas

- Números ao longo do tempo: latência, taxa de erro, uso de CPU/memória.
- Podem alimentar alertas e decisões.
- No laboratório pode ser simples: tempo, status e contadores.

Traces

- Caminho de uma requisição entre serviços.
- Útil para sistemas distribuídos.
- Na disciplina: conceitual ou demonstração curta.

SLI, SLO, SLA e Error Budget

O vocabulário que conecta observabilidade a decisão de negócio

SLI — o indicador medido

SLO — a meta interna

SLA — o compromisso formal

SLI — indicador

- A métrica medida na prática.
- Exemplo: % de requisições respondidas em menos de 300ms.

SLO — objetivo

- A meta interna que o time define para esse indicador.
- Exemplo: 99% das requisições abaixo de 300ms no mês.

SLA e error budget

- SLA: compromisso formal, às vezes contratual, com penalidade.
- Error budget: quanto de falha o SLO tolera — se estourar, prioridade vira estabilidade, não feature nova.

Anatomia de um Incidente

O ciclo que fecha operação -> melhoria

Detecção

- Um alerta dispara, por métrica ou por log.
- MTTD (mean time to detect): tempo até perceber o problema.

Resposta

- Diagnosticar a causa.
- Aplicar mitigação, mesmo que temporária.
- MTTR (mean time to restore): tempo até resolver.

Post-mortem sem culpa

- Documentar o que aconteceu, por que e o que preveniria.
- Objetivo é aprendizado do sistema, não achar um culpado.

Incidente Simulado com IA

Fechar o ciclo operação -> melhoria

Preparar falha

- Quebrar variável de ambiente.
- Subir dependência indisponível.
- Provocar erro em endpoint ou teste de saúde.

Investigar

- Coletar logs e status do container/pipeline.
- Pedir à IA hipóteses e validações.
- Não aceitar correção antes de confirmar a causa.

Resolver e aprender

- Aplicar correção pequena.
- Rodar validação.
- Atualizar README, runbook ou alerta.
- Registrar causa, ação e prevenção.

Entrega/evidência ao final

- Mini relatório de incidente: sintoma, hipótese, evidência, causa, correção, prevenção e papel da IA, se usada.

Alertas: Sinal vs Ruído

Para provocar debate sobre fadiga de alerta

Alerta bom

- Acionável: existe uma ação clara a tomar.
- Específico e ligado a impacto real no usuário.

Alerta ruim

- Dispara sem ação clara do que fazer.
- É ignorado com frequência — fadiga de alerta.
- Mede o sintoma errado.

Pergunta para discussão

- Um time que recebe 50 alertas por dia e ignora a maioria — qual é o risco real disso?
- O que vocês fariam para reduzir o ruído sem perder sinal importante?

AIOps: Observabilidade e Resposta a Incidentes com IA

O problema de "sinal vs. ruído" já tem produto de mercado resolvendo

Datadog

- Watchdog AI: detecção de anomalia, causa raiz automática e forecasting.
- Bits AI apoia a investigação durante o incidente.

PagerDuty

- Módulo de AIOps treinado em 86 bilhões de eventos históricos.
- Reduz ruído de alerta em até 91% — transforma 50 alertas em ~3 incidentes reais.

O papel da IA aqui

- Ela correlaciona, prioriza e roteia para a pessoa certa.
- A causa raiz confirmada e a correção continuam sendo decisão humana.

Atividade 6 - Observabilidade e Apresentação Final

Fechar o projeto com dados e defesa técnica

Perguntas guia

- Que dado prova que a entrega realmente funciona?
- Qual foi a causa real do incidente simulado, não só o sintoma?
- O grupo consegue defender cada decisão em poucos minutos?
- Onde a IA ajudou e onde precisou ser corrigida?

Evidências

- Logs/métricas mínimas da aplicação.
- Mini relatório do incidente simulado.
- Apresentação cobrindo repositório, pipeline, container, release e operação.
- Registro consolidado de uso de IA no projeto.

Entrega

- Apresentação de 5 a 7 minutos.
- Repositório final com todas as evidências do projeto integrador.

Conexão com avaliação

- Fecha os 10% de Diagnóstico DevOps e os 5% de Apresentação final da rubrica.

Prompts Úteis - Operação

IA como apoio à investigação, não como diagnóstico definitivo



Logs

- Analise os logs abaixo e liste hipóteses de causa em ordem de probabilidade. Para cada hipótese, diga qual evidência confirma ou descarta.
- Logs: <trecho sem dados sensíveis>.

Runbook

- Transforme esta solução de incidente em um runbook curto com sintomas, comandos de diagnóstico, ação corretiva e validação final.
- Incidente: <resumo>.

Apresentação

- Com base nas evidências abaixo, ajude a montar uma narrativa técnica de 5 minutos: problema, decisões, pipeline, operação, riscos e próximos passos.
- Evidências: <lista>.

Apresentação Final - Roteiro de 5 a 7 Minutos

Ordem padrão para todos os grupos

1. Contexto e diagnóstico

- Qual problema o projeto representa.
- Quais gargalos DevOps foram priorizados.
- O que foi decidido fazer dentro do tempo disponível.

2. Demonstração técnica

- Repositório e PRs.
- Pipeline rodando.
- Aplicação em container/Compose.
- Release ou rollback/plano.
- Logs/métricas/incidente simulado.

3. Defesa crítica

- Tradeoffs e limitações.
- Uso de IA: onde ajudou, onde falhou, como foi validada.
- Próximos passos para evolução real.

Rubrica Final - Perguntas de Avaliação

Perguntas curtas, respondidas com evidências

Projeto

- Como executo do zero?
- Onde está o README?
- Qual decisão técnica foi mais importante?
- O que ficou fora do escopo?

Pipeline/entrega

- O que faz o pipeline falhar?
- Que teste prova comportamento relevante?
- Qual artefato é promovido?
- Como voltar para versão anterior?

Operação/IA

- Que logs/métricas indicam saúde?
- Qual incidente foi simulado?
- Que sugestão de IA foi aceita ou rejeitada?
- Como vocês validaram?

Checklist do Projeto Integrador

Trilha de acompanhamento dos entregáveis

Repositório

- README
- Commits e PRs
- Tags/releases
- Decisões documentadas

Automação

- Pipeline CI
- Testes
- Qualidade
- Artefato/imagem

Execução

- Dockerfile
- compose.yaml
- Variáveis
- Health check

Entrega

- Release
- Rollback
- Segurança mínima

Operação e IA

- Logs
- Métricas
- Incidente simulado
- Hipóteses com IA validadas

Análise e IA

- Diagnóstico
- Gargalos
- Uso de IA registrado
- Validações e riscos
- Evolução proposta

Fechamento da Disciplina

Conectando prática, maturidade e evolução profissional

Síntese

- DevOps é fluxo de entrega com responsabilidade compartilhada.
- Automação sem feedback e evidência vira ritual vazio.
- IA aumenta velocidade, mas também exige validação e governança.
- O valor está em entregar software reproduzível, seguro, observável e evolutivo.

Anexos

Modelos rápidos, comandos de apoio e bibliografia

Template - Registro de Uso de IA

Um registro simples, não burocrático

Contexto

- Ferramenta usada.
- Objetivo do uso.
- Trecho/contexto fornecido, sem dados sensíveis.

Resultado

- Saída útil recebida.
- O que foi aceito.
- O que foi rejeitado ou corrigido.

Validação

- Teste, build, revisão ou execução feita.
- Risco identificado.
- Decisão humana final.

Entrega/evidência ao final

- Um parágrafo ou tabela no README/relatório final basta, desde que permita defender a decisão.

Template - Diagnóstico DevOps em 1 Página

Entregável do Encontro 1

Diagnostico DevOps

Projeto/grupo:

Fluxo atual:

1. Pedido/demanda
2. Desenvolvimento
3. Teste
4. Entrega
5. Operacao

Gargalos:

- <gargalo 1>
- <gargalo 2>
- <gargalo 3>

Priorizacao

Melhoria escolhida:

Impacto esperado:

Evidencia de sucesso:

Riscos:

IA pode ajudar em:

Validacao humana necessaria:

Template — Entrega Inicial do Projeto

Registro do incremento do Encontro 2 no próprio repositório

Identificação

Projeto e objetivo
Organização e URL do repo
Integrantes e responsáveis
Acessos confirmados

Base inicial

Hash do primeiro commit
Stack e funcionalidade mínima
Comando de execução
Comando e resultado do teste

Validação

Quem validou o clone limpo
Link/log da evidência
IA: uso e validação, se aplicada
Próximo passo: pipeline CI

A evidência conecta pessoas, código, execução e o próximo incremento.

Template - Evidência de Pipeline CI

Entregável do Encontro 3

Evidencia de Pipeline CI

Projeto/grupo:

Trigger utilizado: push / pull_request

Execução

Link ou print do pipeline:

Status: verde / vermelho

Teste que prova o comportamento

Nome do teste:

O que ele valida:

Falha analisada (se houve)

Causa:

Correção aplicada:

Uso de IA (se aplicável)

Hipótese sugerida:

Validação humana feita:

Template - Checklist de Containerização

Entregável do Encontro 4

Checklist de Containerizacao

Projeto/grupo:

Execucao

Comando de subida: docker compose up --build

Porta exposta:

Variaveis obrigatorias:

Evidencia

Print/log da aplicacao respondendo:

Endpoint testado:

Falha diagnosticada (se houve)

Sintoma:

Causa:

Correcao:

Template - Mini Relatório de Incidente

Entregável do Encontro 6

Mini Relatório de Incidente

Projeto/grupo:

Sintoma observado:

Hipótese inicial:

Evidência coletada (log/métrica):

Causa confirmada:

Correção aplicada:

Prevenção futura:

Papel da IA (se usada):

- Hipótese sugerida:
- Validação humana:

Comandos Rápidos para Consulta

Referência para destravar o laboratório

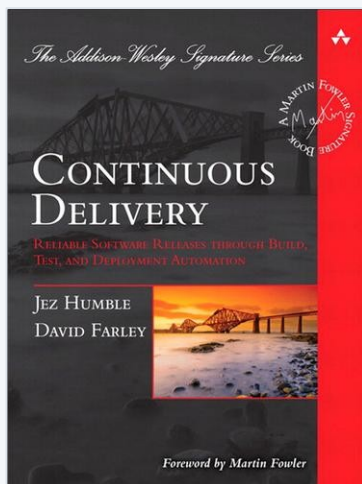
```
# Git
git status
git diff
git switch -c feature/x
git add .
git commit -m "tipo: mensagem"
git push -u origin feature/x
git pull
git tag v0.1.0
git log --oneline --graph --decorate

# CI/CD (conferir local antes do pipeline)
npm ci
npm test
npm run build

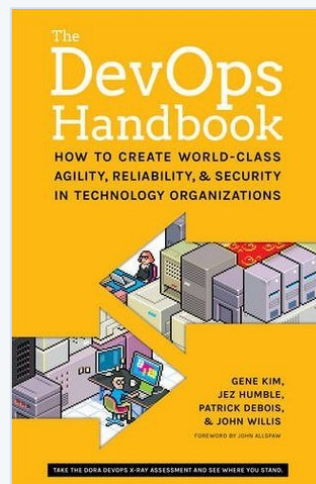
# Docker/Compose
docker build -t app:local .
docker run --rm -p 3000:3000 app:local
docker compose up --build
docker compose logs -f app
docker compose ps
docker compose down -v
curl http://localhost:3000/health
```

Bibliografia Básica

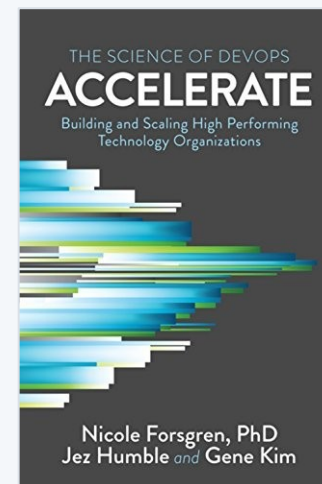
Os quatro livros de referência do plano de ensino



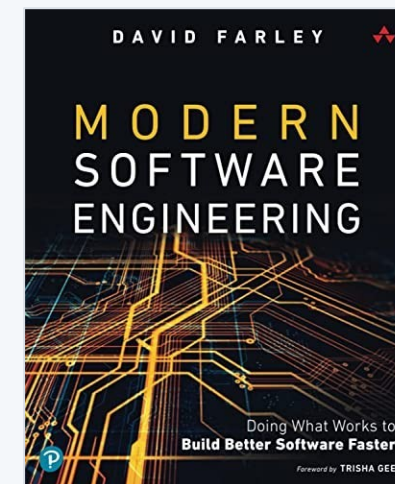
HUMBLE, J.; FARLEY, D.
Continuous Delivery
Addison-Wesley, 2010



KIM, G. et al.
The DevOps Handbook
2. ed. IT Revolution, 2021



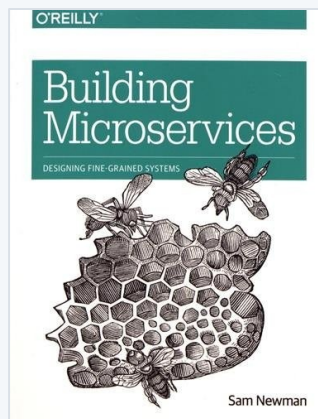
FORSGREN, N.; HUMBLE, J.; KIM, G.
Accelerate
IT Revolution, 2018



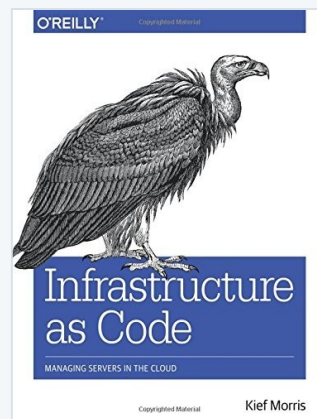
FARLEY, D.
Modern Software Engineering
Addison-Wesley, 2021

Bibliografia Complementar

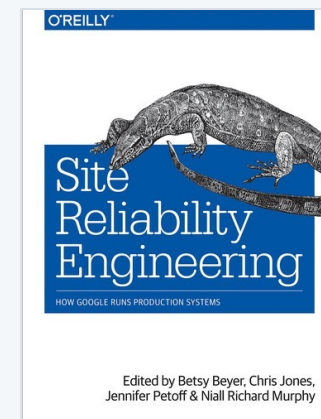
Aprofundamento em microsserviços, infraestrutura e operação



NEWMAN, S.
Building Microservices
2. ed. O'Reilly, 2021



MORRIS, K.
Infrastructure as Code
2. ed. O'Reilly, 2020



BEYER, B. et al. (org.)
Site Reliability Engineering
O'Reilly, 2016

Ferramentas, métricas e governança de IA — referências online

- DORA; Google Cloud. DORA's software delivery performance metrics.
- GitHub. GitHub Actions documentation. | GitHub. GitHub Copilot documentation.
- Docker. Docker Compose documentation. | HashiCorp. Terraform Documentation.
- OpenAI. Codex: AI coding agents for software engineering. | Google. Gemini Code Assist overview.
- NIST. Artificial Intelligence Risk Management Framework (AI RMF 1.0), 2023.
- OWASP. OWASP GenAI LLM Top 10 2026.